Interactive 3d Pinball Game

Computer Graphics and Image Processing Laboratory

CSE 4102

Submitted by

Al Nahian Zarif

Roll: 2107098

4th Year

Contents

1 Introduction 1

2 Verified Feature Overview 2

2.1 Project Scope 3

2.2 Verification Boundaries 3

3 Coordinate System and Mathematical Foundation 4

3.1 World and Local Coordinates 4

3.2 Point and Direction Vector 5

3.3 Dot Product Cross Product and Normalization 5

4 Transformation Mathematics 6

4.1 Translation 6

4.2 Scaling 6

4.3 Rotation 7

4.4 Model Matrix and Parent Placement 8

4.5 Example of the Inclined Playfield 8

4.6 Worked Cuboid and Leg Coordinates 9

5 Procedural Geometry 10

5.1 Master Cuboid 10

5.1.1 Face Normals and Triangle Assembly 11

5.2 Plane and Inclined Board 12

5.3 Cylinder 13

5.3.1 Worked Bumper Cylinder 14

5.4 Sphere 15

5.4.1 Sphere Indices and Ball Vertices 16

5.5 Flipper Body and Cap Geometry 17

5.6 Torus Ring 18

5.7 Helical Spring Factory 19

5.8 Text and Television Screen 19

5.9 Beanbags and Custom Rails 20

5.10 Rails Derived from Collision Segments 20

6 Scene Construction and Reusable Meshes 21

6.1 GPU Representation 22

7 Pinball Hierarchy and Flipper Kinematics 23

7.1 Flipper Dimensions and Stroke 23

7.1.1 Tip Direction 24

7.2 Parent and Child Transform Chains 24

7.3 Fan and Door Rigid Assemblies 24

Contents continued

8 Ball Dynamics and Gameplay 25
8.1 Launch States and Plunger 25
8.2 Fixed Step Integration and Surface Constraint 26
8.3 Scoring Events and Contact Cooldowns 27
8.4 Bonus Multipliers Ball Saves and Lives 28
9 Lighting Theory and Implementation 29
9.1 Phong Lighting Vectors 29
9.2 Ambient Diffuse and Specular Terms 29
9.3 Implemented Color Contributions 30
9.4 Distance Attenuation 30
9.5 Spotlight Cone Factor 30
9.6 Effective Light Rig 31
9.7 Day Night and Room Light Control 32
10 Materials and Shading 33
10.1 Material Parameters 33
10.2 Lighting Model and Shading Method 34
10.2.1 Flat Mode in This Implementation 34
10.2.2 Gouraud Shading 34
10.2.3 Phong Shading 34
10.3 Visual Comparison of Shading Modes 35
11 Normal Transformation 36
12 Camera System 37
12.1 Room and Table Cameras 37
12.2 Perspective Projection 38
12.3 Movement Zoom and Picking Rays 38
13 Animation and Time Based Motion 39
13.1 Ceiling Fan Acceleration and Coasting 39
13.2 Hinged Refrigerator Door 39
13.3 Pinball Animation and Event Feedback 40
14 Collision Detection and Contact Response 41
14.1 Ball and Circular Bumper 41
14.2 Ball and Rectangular Target 41
14.3 Ball and Finite Rail Segment 42
14.4 Moving Flipper Contact 42
14.5 Walls Drain and Numerical Safeguards 43
15 Gaming Lounge and Ray Tracing 44

Contents continued

15.1 Lounge Layout 44
15.2 Ray Scene Snapshot and BVH 45
15.3 Ray Box and Triangle Intersections 46
15.4 Surface Attributes at a Hit 46
15.5 Shadows and Mirror Reflection 47
15.6 Refraction Fresnel and Total Internal Reflection 48
15.7 Ray Traced Scene Results 49
15.8 Live Rendering and Quality Controls 50
16 Interaction and Runtime Controls 51
16.1 Console Buttons and Scene Picking 52
17 Rendering Flow 53
17.1 Opaque Transparent and HUD Passes 54
17.2 Procedural Textures 54
17.3 Ray Presentation Pass 54
18 Implementation Summary and Validation 55
18.1 Implementation Map Continued 56
18.2 Build and Regression Results 57
18.3 Rendered Capture Verification 57
19 Conclusion 58
19.1 Limitations and Future Work 58
References 59
Report Preparation 59
Appendix A Coordinate Registry 60
A.1 Bumpers and Targets 60
A.2 Ready Ball and Flipper Tips 60
A.3 Complete Rail Endpoint Registry 61
A.4 Cube Corner Rule for All Box Objects 61
Appendix B Complete Vertex Archive 62
B.1 Export Schema and Snapshot Conditions 62

List of Figures

Figure 1.1: Actual pinball gameplay capture showing the playfield, ball and score console. 1

Figure 2.1: Actual gaming-lounge capture with room controls and the pinball machine at the side. 3

Figure 3.1: Coordinate conventions for the machine and the room. 4

Figure 5.1: The eight cuboid corner labels used by the source. 11

Figure 5.2: Inclined board surface and the vertically seated ball center. 12

Figure 5.3: Cylinder rings, caps, radius and local Z axis. 13

Figure 5.4: Latitude and longitude construction of the steel-ball sphere. 15

Figure 5.5: Two indexed triangles from one sphere grid cell. 16

Figure 5.6: Flipper dimensions and the relationship between rendered geometry and contact geometry. 17

Figure 5.7: Torus geometry used by the reactor insert and bumper skirts. 18

Figure 6.1: Vertex attributes, buffer storage and the two drawing paths. 22

Figure 7.1: Opposing flippers at their rest and active angles. 23

Figure 7.2: Machine placement and the independent animated room assemblies. 24

Figure 8.1: Ball launch, play, drain and restart state transitions. 25

Figure 9.1: Vectors used by the project's Phong diffuse and specular calculation. 29

Figure 9.2: Inner and outer cone limits for the table spotlight. 30

Figure 9.3: Room lighting enabled in the Phong raster view. 32

Figure 9.4: Actual room capture with room lights disabled. 32

Figure 10.1: Mode labeled Flat with ambient and diffuse lighting. 35

Figure 10.2: Gouraud mode with lighting evaluated at vertices. 35

Figure 10.3: Phong mode with lighting evaluated per fragment. 35

Figure 11.1: Correct and incorrect normal directions after a nonuniform transform. 36

Figure 12.1: Default room and table camera states after machine placement. 37

Figure 13.1: Idealized time profiles defined by the fan and door update rules. 39

Figure 13.2: Flippers and lower-board artwork from the actual game capture. 40

Figure 14.1: Closest-point contact between the ball and a finite rail segment. 42

Figure 15.1: Schematic plan of the gaming lounge and machine placement. 44

Figure 15.2: Bounding volume hierarchy used to accelerate triangle intersection. 45

Figure 15.3: Primary, bounded shadow and recursive reflection rays. 47

Figure 15.4: Reflection and refraction at the two interfaces of a glass pane. 48

Figure 15.5: Fresh room ray-tracing capture at a 960-pixel ray width and four samples per pixel. 49

Figure 15.6: Actual ray-tracing capture with the refrigerator door open. 49

Figure 17.1: Raster pipeline from generated vertices to the final framebuffer. 53

List of Tables

Table 2.1: Implemented project features and their source locations. 2

Table 3.1: Coordinate and numerical conventions. 5

Table 4.1: Derived board extents after the 8 degree rotation. 8

Table 4.2: All cabinet corner positions in machine coordinates. 9

Table 5.1: All eight unique local cuboid corners. 10

Table 5.2: Exact cuboid face order from createCube. 11

Table 5.3: Plane positions and texture coordinates. 12

Table 5.4: Main-bumper cylinder ring samples. 14

Table 5.5: Selected cylinder instances. 14

Table 5.6: Selected exact ball-sphere samples from the indexed grid. 16

Table 5.7: Reactor-ring samples with $R = 0.68$ and $r = 0.04$. 18

Table 5.8: Television-screen quad from createScreenMesh(2.98,1.77). 19

Table 6.1: Principal machine components in simulation coordinates. 21

Table 6.2: Vertex attributes and GPU bindings. 22

Table 7.1: Flipper state and movement parameters. 23

Table 8.1: Scoring rules before applying the current multiplier. 27

Table 8.2: State changes for a save and an ordinary lost ball. 28

Table 9.1: Effective room point-light configuration. 31

Table 9.2: Active table spotlight values. 31

Table 9.3: Scene ambient RGB under the four room-light conditions. 32

Table 10.1: Representative material values used by the source. 33

Table 13.1: Time-dependent update paths. 40

Table 14.1: Robustness measures in contact and interaction code. 43

Table 15.1: Selected room-world object positions. 44

Table 15.2: Ray-rendering controls and implementation limits. 50

Table 16.1: Runtime keyboard and mouse controls from main.cpp. 51

Table 16.2: Room console action IDs. 52

Table 18.1: Mathematics mapped to the principal implementation. 55

Table 18.2: Interaction rendering and ray-tracing implementation map. 56

Table 18.3: Observed regression-test run. 57

Table A.1: Exact object placement registry rounded to six decimals. 60

Table A.2: All eleven physical and visible rail segments. 61

Table B.1: Geometry archive inventory. 62

Chapter 1

Introduction

Interactive 3d Pinball Game is a C++17 and OpenGL 3.3 application that combines a playable pinball machine with an interactive gaming lounge. The player launches a steel ball, operates two flippers, hits bumpers and drop targets, collects score and bonus, and manages three balls per game. The surrounding room adds movable cameras, switchable lighting, an animated ceiling fan, a hinged refrigerator door and a CPU ray-tracing mode.

The project connects the main topics of computer graphics to visible behavior. Procedural equations generate meshes; model, view and projection matrices position them; normal transformation and lighting determine their appearance; time-based updates animate the scene; and collision tests drive gameplay. This report follows those connections from vertex construction to final pixels and user interaction.

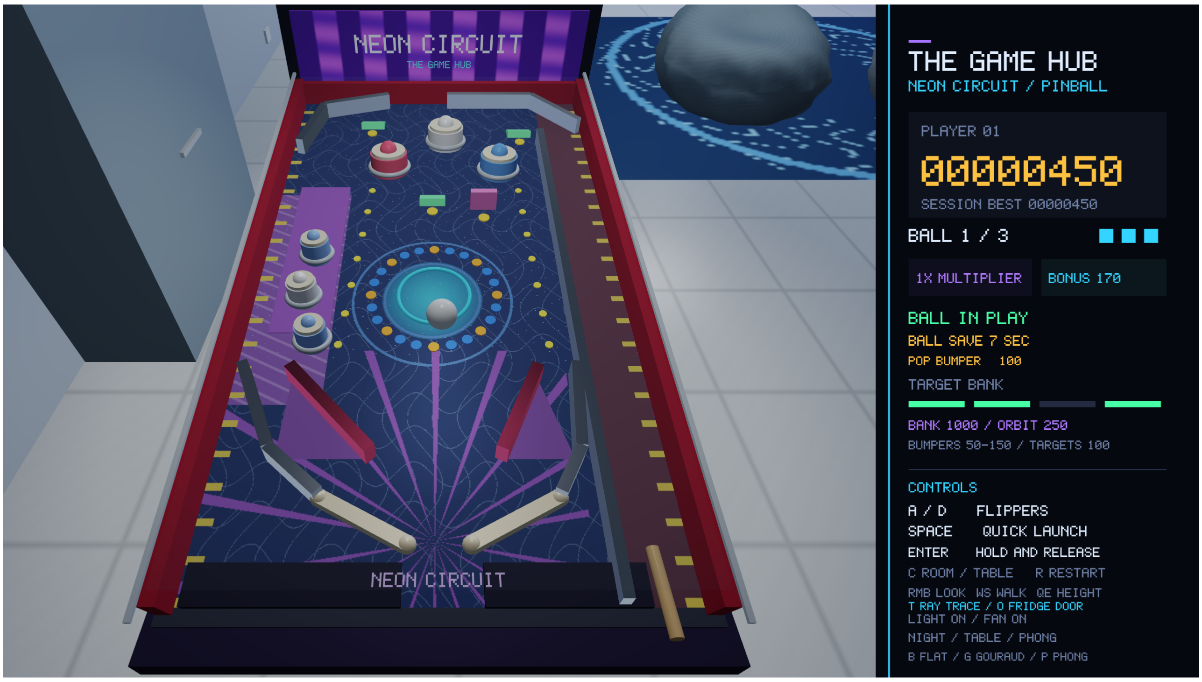

NEON CIRCUIT
THE GAME HUB
NEON CIRCUIT
THE GAME HUB
NEON CIRCUIT / PINBALL
PLAYER 01
00000450
SESSION BEST 00000450
BALL 1 / 3
1X MULTIPLIER
BONUS 170
BALL IN PLAY
BALL SAVE 7 SEC
POP BUMPER 100
TARGET BANK
BANK 1000 / ORBIT 250
BUMPERS 50-150 / TARGETS 100
CONTROLS
A / D FLIPPERS
SPACE QUICK LAUNCH
ENTER HOLD AND RELEASE
C ROOM / TABLE R RESTART
RMB LOOK WS WALK QE HEIGHT
T RAY TRACE / O FRIDGE DOOR
LIGHT ON / FAN ON
NIGHT / TABLE / PHONG
B FLAT / G GOURAUD / P PHONG

Figure 1.1: Actual pinball gameplay capture showing the playfield, ball and score console.

The report is based on the current source files, rendered project images and regression tests. Numerical values refer to simulation coordinates unless a room-world placement is explicitly stated. The complete vertex export accompanies the report as a geometry archive.

Chapter 2

Verified Feature Overview

Table 2.1: Implemented project features and their source locations.

Feature	Verified behavior	Source
Procedural geometry	Cuboids, planes, cylinders, spheres, flippers, rings, text and custom beanbag meshes.	Mesh.cpp; GameHub.cpp
Transformations	Translation, quaternion rotation, nonuniform scale and machine placement.	Transform.cpp; GameHub.cpp
Pinball physics	Fixed 240 Hz steps, board slope, damping, rail and moving-flipper contacts.	PinballMachine.cpp; Ball.cpp
Game rules	Ready/shooter/in-play states, three balls, ball save, bonus and multipliers.	PinballMachine.cpp
Lighting	Seven effective room point lights, one table spotlight and day/night ambient.	GameHub.cpp; shaders
Shading	Selectable Flat, Gouraud and Phong shader programs.	main.cpp; shaders
Room animation	Fan acceleration/coasting and reversible refrigerator-door animation.	GameHub.cpp
Ray tracing	Triangle BVH, shadows, reflection, refraction, Fresnel and CPU workers.	RayTracer.cpp; RayTraceView.cpp
Interaction	Keyboard controls, console buttons and ray-based scene picking.	main.cpp; HUD.cpp
Verification	Three headless CTest suites for physics, room controls and ray tracing.	tests/; CMakeLists.txt

2.1 Project Scope

The machine uses six bumpers, four drop targets, two flippers and one shooter lane. A launch enters the upper board through guide rails. During play, the ball can travel through a left-orbit region or strike slingshot rails. The console reports score, session best, current ball, remaining balls, multiplier, pending bonus, target-bank progress and ball-save time.

The lounge contains white tiled flooring, navy wall panels, blue LED borders, a television and console table, two beanbags on a patterned rug, cabinets, a beverage refrigerator, pendant lights, a separate ceiling fan, a mascot, an air conditioner and a glass side wall. Most components are generated from procedural primitives.

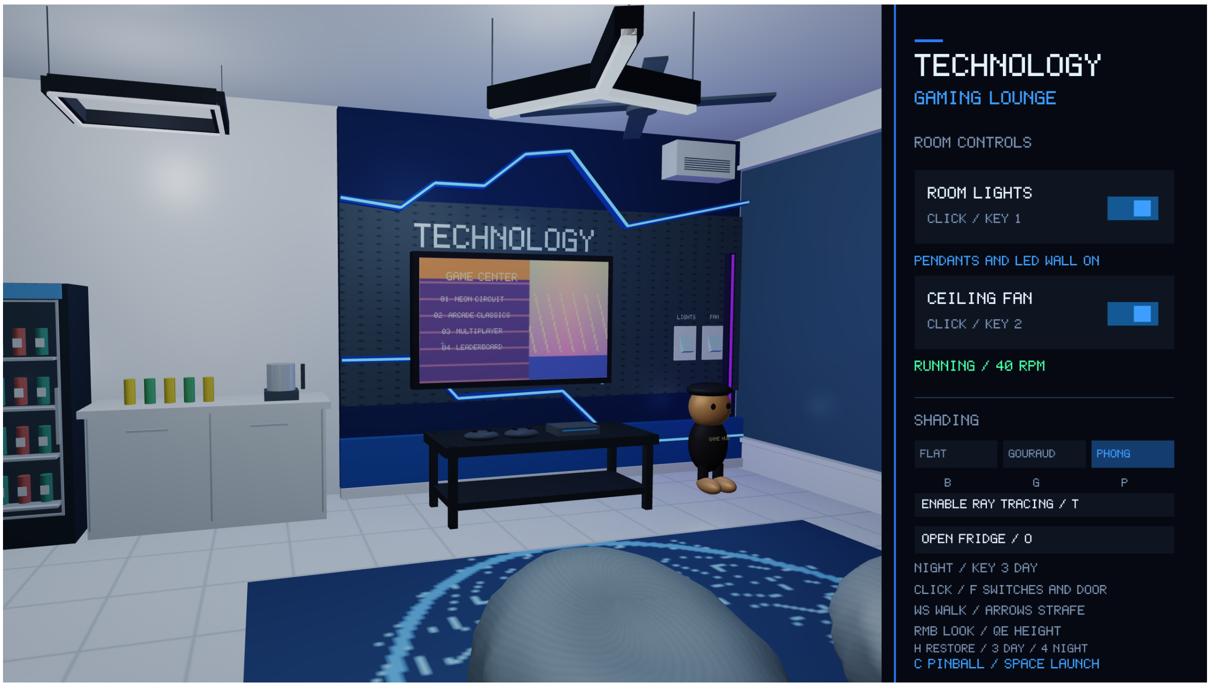

TECHNOLOGY
GAME CENTER
01 NEON CIRCUIT
02 ARCADE CLASSICS
03 MULTIPLAYER
04 LEADERBOARD
LIGHTS
FAN
TECHNOLOGY
GAMING LOUNGE
ROOM CONTROLS
ROOM LIGHTS
CLICK / KEY 1
PENDANTS AND LED WALL ON
CEILING FAN
CLICK / KEY 2
RUNNING / 40 RPM
SHADING
FLAT
GOURAUD
PHONG
B
G
P
ENABLE RAY TRACING / T
OPEN FRIDGE / O
NIGHT / KEY 3 DAY
CLICK / F SWITCHES AND DOOR
WS WALK / ARROWS STRAFE
RMB LOOK / QE HEIGHT
H RESTORE / 3 DAY / 4 NIGHT
C PINBALL / SPACE LAUNCH

Figure 2.1: Actual gaming-lounge capture with room controls and the pinball machine at the side.

2.2 Verification Boundaries

Source inspection establishes what is implemented. Tests check selected behavioral properties, and captures show representative visual output. A successful test run is not a benchmark of all hardware or every possible input sequence. The implementation-specific limitations of flat shading, surface-constrained physics and ray tracing are stated in the relevant chapters.

Chapter 3

Coordinate System and Mathematical Foundation

3.1 World and Local Coordinates

The project uses a right-handed, Z-up coordinate system. On the pinball board, X runs across the width, Y runs along the board toward the backbox, and Z gives height. The gaming lounge also uses Z as its vertical axis. This differs from examples that assume Y is the vertical direction, so every rotation and camera basis must be read using the project convention.

$$p_{\text{local}} = (x, y, z, 1)^T \; ; \; p_{\text{world}} = M_{\text{scene}} \, M_{\text{object}} \, p_{\text{local}} \tag{3.1}$$

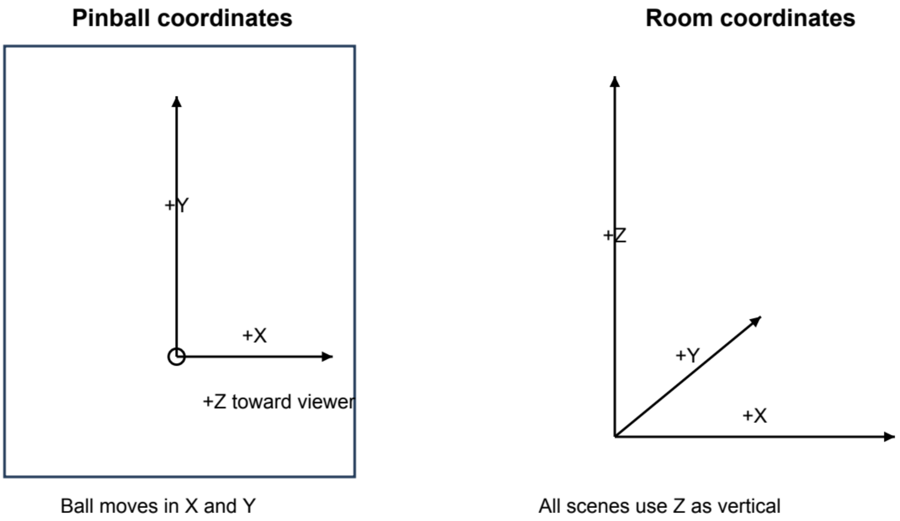

Pinball coordinates
+Y
+X
+Z toward viewer
Ball moves in X and Y
Room coordinates
+Z
+Y
+X
All scenes use Z as vertical

Figure 3.1: Coordinate conventions for the machine and the room.

A mesh stores local positions. Its object transform provides size, orientation and position. The complete machine receives an additional scene placement when drawn in the room. Room scenery uses the identity scene transform. Keeping machine coordinates separate preserves the physics rules when the machine is resized for the lounge.

3.2 Point and Direction Vector

Homogeneous coordinates distinguish a point from a direction. A point has w = 1 and receives translation; a direction has w = 0 and does not. This distinction is used when converting a picking ray into the moving refrigerator door's local frame.

$$\text{point} = (x, y, z, 1)^T \text{ ; direction} = (d_x, d_y, d_z, 0)^T \tag{3.2}$$

3.3 Dot Product Cross Product and Normalization

$$a \cdot b = a_x b_x + a_y b_y + a_z b_z \tag{3.3}$$

$$|v| = \sqrt{(v \cdot v)} \text{ ; } \hat{v} = v / |v| \tag{3.4}$$

Dot products measure projections and angular alignment. Collision code projects a ball onto a segment and measures its incoming normal velocity. Lighting uses N · L for diffuse brightness and V · R for specular highlights. Cross products build camera right/up directions, face normals and rail side vectors.

Table 3.1: Coordinate and numerical conventions.

Quantity	**Value or convention**
Playfield width and projected length	7.0 and 12.0 simulation units
Playfield center height and tilt	1.4 units and 8 degrees
Ball radius	0.28 simulation units
Room size	12 x 14 x 5.6 room units
Machine room placement	Origin (-3.5, -3.8, 0.45); uniform scale 0.28
Angles in object state	Degrees; converted to radians for trigonometry and GLM
Vector safeguards	Zero-length and near-parallel cases are checked before division

The unit names are mathematical scene units rather than certified real-world dimensions. The numerical gravity constant is 9.81, but the chosen machine scale is a presentation transform rather than a physical unit conversion.

Chapter 4

Transformation Mathematics

4.1 Translation

Translation adds a position offset to a local point. For a column-vector point, the translation values occupy the last column of the matrix. The following matrices are written by rows; GLM stores matrix data in column-major order.

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_\gamma \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.1}$$

$$x' = x + t_x \; ; y' = y + t_y \; ; z' = z + t_z \tag{4.2}$$

4.2 Scaling

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_\gamma & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.3}$$

Nonuniform scale gives different factors along the three axes. It creates elongated controller bodies, mascot parts and differently shaped beanbags. The complete machine uses a uniform factor of 0.28 in the lounge. A positive scale changes size without reversing orientation.

4.3 Rotation

Positive rotations follow the right-hand rule. The board tilts about X; flippers, fan blades and the refrigerator hinge rotate about Z. For θ in radians, the conventional elementary rotation matrices are:

$$R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.4}$$

$$R_y = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.5}$$

$$R_z = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.6}$$

General GameObject rotation is formed with **glm::quat(glm::radians(rotation))** and converted to a 4 x 4 matrix. Therefore the authoritative model transform is the quaternion-derived rotation from the stored Euler-angle vector. The explicit single-axis matrices above directly describe the principal animated components.

4.4 Model Matrix and Parent Placement

Transform::getModelMatrix begins with identity, applies translation, multiplies by the quaternion rotation and then applies scale. With column vectors, the rightmost operation acts first:

$$M_{object} = T\,R_{quat}\,S \tag{4.7}$$

```
model = glm::translate(model, position);
model = model * glm::mat4_cast(quat);
model = glm::scale(model, scale);
```

$$M_{machine} = T(-3.5, -3.8, 0.45)\ S(0.28, 0.28, 0.28) \tag{4.8}$$

$$p_{room} = (-3.5, -3.8, 0.45) + 0.28\ p_{machine} \tag{4.9}$$

4.5 Example of the Inclined Playfield

The visible plane has width 7 and local length 12 / cos 8°. It is translated to Z = 1.4 and rotated by 8° about X. This compensating local length preserves a projected Y extent of exactly 12 after rotation.

$$M_{board} = T(0, 0, 1.4)\ R_x(8°) \tag{4.10}$$

$$z(y) = 1.4 + y\tan(8°) \tag{4.11}$$

Table 4.1: Derived board extents after the 8 degree rotation.

Quantity	**Computed value**
Local half length	6 / cos 8° = 6.058965
World Y limits	-6.000000 and +6.000000
Surface Z at Y = -6	0.556755
Surface Z at Y = 0	1.400000
Surface Z at Y = +6	2.243245

4.6 Worked Cuboid and Leg Coordinates

The cabinet is a cuboid of size (7.8, 13.3, 0.4) centered at (0, 0, 0.22). Its eight machine-space corners are obtained by adding every sign combination of half dimensions to the center. The four chrome legs have size (0.15, 0.15, 0.5) and centers (±3.5, ±5.7, 0.25).

$$\text{corner} = \text{center} + (\pm w/2, \pm h/2, \pm d/2) \tag{4.12}$$

Table 4.2: All cabinet corner positions in machine coordinates.

Corner	**X**	**Y**	**Z**
v0	-3.900	-6.650	0.020
v1	3.900	-6.650	0.020
v2	3.900	6.650	0.020
v3	-3.900	6.650	0.020
v4	-3.900	-6.650	0.420
v5	3.900	-6.650	0.420
v6	3.900	6.650	0.420
v7	-3.900	6.650	0.420

For the front-left leg, local v0 = (-0.075, -0.075, -0.25). Translating by its center (-3.5, -5.7, 0.25) gives (-3.575, -5.775, 0). Applying the room placement gives (-4.501, -5.417, 0.45). Its opposite corner v6 becomes (-3.425, -5.625, 0.5) in machine coordinates and (-4.459, -5.375, 0.59) in room coordinates.

The same equations generate the other three legs by replacing the center signs. The room-world scale does not change the machine-space collision radius or launch speed. The transformation hierarchy controls drawing and the ray-scene export.

Chapter 5

Procedural Geometry

Each Vertex contains a position, normal, color and texture coordinate. Triangles either reference stored vertices through an index vector or directly occupy consecutive groups of three records. The project uses both representations. A geometric corner count is therefore different from the number of GPU vertex records.

5.1 Master Cuboid

Source: Mesh.cpp - createCube(). For width w, height h and depth d, the mesh is centered at the origin. A unit cube has half dimensions 0.5.

Table 5.1: All eight unique local cuboid corners.

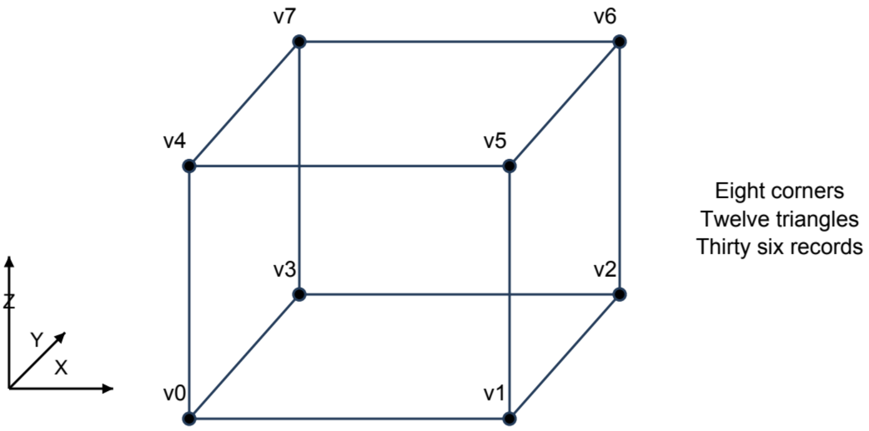
ID	General position	Unit cube position
v0	(-w/2, -h/2, -d/2)	(-0.5, -0.5, -0.5)
v1	(w/2, -h/2, -d/2)	(0.5, -0.5, -0.5)
v2	(w/2, h/2, -d/2)	(0.5, 0.5, -0.5)
v3	(-w/2, h/2, -d/2)	(-0.5, 0.5, -0.5)
v4	(-w/2, -h/2, d/2)	(-0.5, -0.5, 0.5)
v5	(w/2, -h/2, d/2)	(0.5, -0.5, 0.5)
v6	(w/2, h/2, d/2)	(0.5, 0.5, 0.5)
v7	(-w/2, h/2, d/2)	(-0.5, 0.5, 0.5)

createCube expands six faces into two triangles each, storing 36 vertex records with no index entries. The faces have separate normal attributes, so a corner can appear several times with different normals. This representation gives hard edges on cabinet, wall and furniture faces.

5.1.1 Face Normals and Triangle Assembly

Table 5.2: Exact cuboid face order from createCube.

Face	Ordered corners	Assigned normal	Triangles
Front	0, 1, 2, 3	(0, 0, -1)	0-1-2; 0-2-3
Back	4, 7, 6, 5	(0, 0, 1)	4-7-6; 4-6-5
Left	0, 4, 7, 3	(-1, 0, 0)	0-4-7; 0-7-3
Right	1, 5, 6, 2	(1, 0, 0)	1-5-6; 1-6-2
Bottom	0, 4, 5, 1	(0, -1, 0)	0-4-5; 0-5-1
Top	3, 7, 6, 2	(0, 1, 0)	3-7-6; 3-6-2



Eight corners
Twelve triangles
Thirty six records

Figure 5.1: The eight cuboid corner labels used by the source.

The corner names and normal directions above reproduce the source exactly. Some faces have triangle winding that disagrees with their assigned outward normal. The raster setup does not enable face culling, and the ray exporter aligns geometric normals with the summed vertex normals. This is relevant to dielectric entry/exit classification.

Face names follow the factory comments. In the Z-up room, the physically upper face is the source's Back face with normal (0,0,1); Top names the +Y face.

Face texture coordinates follow (0,0), (1,0), (1,1), (0,1). Expanded triangle records use UV order 0-1-2 and 0-2-3 for each face. The complete 36-record unit-cube data is included in cube_unit_vertices.csv.

5.2 Plane and Inclined Board

Source: Mesh.cpp - createPlane(); Playfield.cpp. A plane lies in XY with Z = 0. It has four unique corners and six expanded records, using triangles (0,1,2) and (0,2,3). Every normal is (0,0,1).

Table 5.3: Plane positions and texture coordinates.

Corner	**Local position**	**UV for repeat q**
v0	(-w/2, -d/2, 0)	(0, 0)
v1	(w/2, -d/2, 0)	(q, 0)
v2	(w/2, d/2, 0)	(q, q)
v3	(-w/2, d/2, 0)	(0, q)

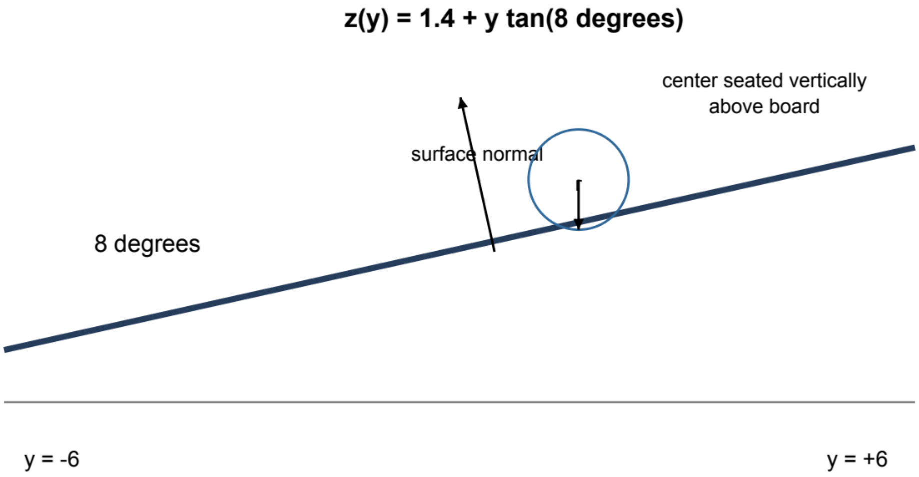

z(y) = 1.4 + y tan(8 degrees)
center seated vertically above board
surface normal
8 degrees
y = -6
y = +6

Figure 5.2: Inclined board surface and the vertically seated ball center.

The board uses q = 1 so the artwork covers it once. The room floor uses createPlane(12,14,3.5), repeating the tile texture. The playfield's rigid rotation gives normal (0,-sin 8°,cos 8°). Ball seating instead sets center Z to the surface height plus radius; it is a vertical offset rather than an exact perpendicular-distance constraint.

$$N_{\text{board}} = (0, -0.139173, 0.990268) \tag{5.1}$$

5.3 Cylinder

Source: Mesh.cpp - createCylinder(). The cylinder axis is local Z, with caps at Z = ±h/2. For N angular segments, the angle at ring sample i is:

$$\theta_i = 2\pi i/N \;;\; x_i = r \cos \theta_i \;;\; y_i = r \sin \theta_i \tag{5.2}$$

$$p_{i,\pm} = (r \cos \theta_i, r \sin \theta_i, \pm h/2) \tag{5.3}$$

$$N_{side} = (\cos \theta, \sin \theta, 0) \;;\; N_{caps} = (0, 0, \pm 1) \tag{5.4}$$

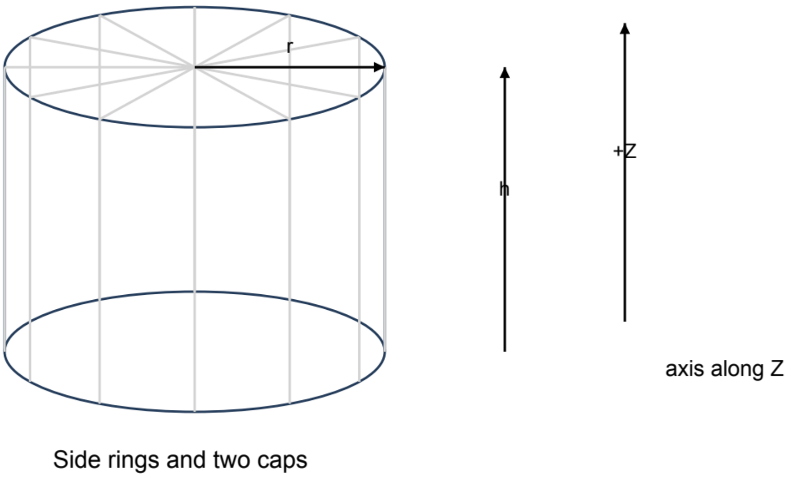


Figure 5.3: Cylinder rings, caps, radius and local Z axis.

Each segment produces two side triangles, one top-cap triangle and one bottom-cap triangle. Each triangle is expanded into three records, so a cylinder contains 12N records and 4N triangles. Side UVs are (i/N,0) and (i/N,1); cap vertices use the default zero UV values.

$$V = 12N \;;\; T = 4N \;;\; \text{EBO entries} = 0 \tag{5.5}$$

5.3.1 Worked Bumper Cylinder

A main bumper has r = 0.42, h = 0.45 and N = 28. Its caps lie at Z = ±0.225. The numerical table samples the exact rings at quarter-turn positions; both cap signs use the same X and Y.

Table 5.4: Main-bumper cylinder ring samples.

i	θ degrees	X	Y	Z levels
0	0	0.420000	0.000000	±0.225000
1	12.857143	0.409470	0.093459	±0.225000
7	90	0.000000	0.420000	±0.225000
14	180	-0.420000	0.000000	±0.225000
21	270	0.000000	-0.420000	±0.225000
28	360	0.420000	0.000000	±0.225000

The same factory creates the plunger, fan hub, fan rod, beverage bottles, kettle, ceiling downlights, illuminated lane inserts and bumper caps. A plunger cylinder is rotated 90° about X so its axis lies along the shooter lane rather than vertically.

Table 5.5: Selected cylinder instances.

Object	Radius	Height	Segments	Records
Main bumper	0.42	0.45	28	336
Side bumper	0.28	0.45	28	336
Plunger	0.09	1.15	18	216
Fan hub	0.17	0.16	24	288
Beverage bottle	0.062	0.30	12	144
Lane lamp	0.075	0.025	10	120

The bumper mesh radius is also used by XY collision detection. Decorative skirts and caps are additional meshes and do not enlarge the bumper's physical contact radius.

5.4 Sphere

Source: Mesh.cpp - createSphere(). Latitude φ covers -π/2 to +π/2, and longitude θ covers one full turn. For L latitude segments and K longitude segments:

$$\varphi_i = -\pi/2 + \pi i/L \; ; \; \theta_j = 2\pi j/K \tag{5.6}$$

$$p(\varphi,\theta) = (r \cos\varphi \cos\theta, r \cos\varphi \sin\theta, r \sin\varphi) \tag{5.7}$$

$$N = p/r \; ; \; UV = (j/K, i/L) \tag{5.8}$$

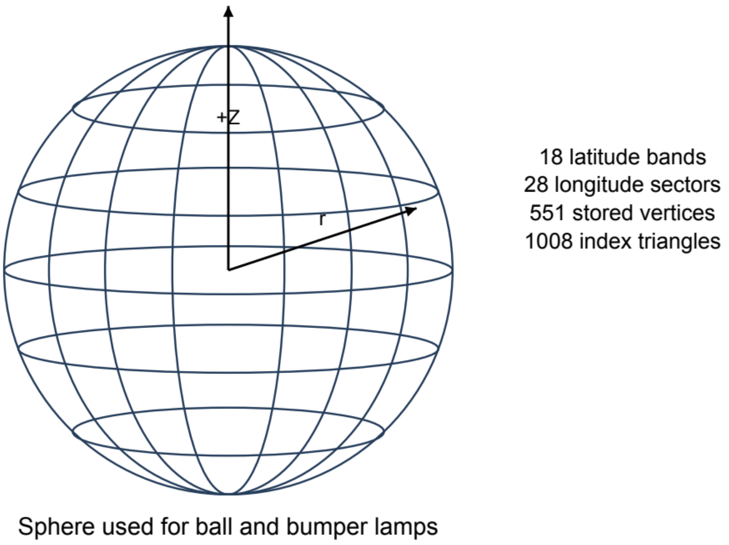

+Z
r
18 latitude bands
28 longitude sectors
551 stored vertices
1008 index triangles
Sphere used for ball and bumper lamps

Figure 5.4: Latitude and longitude construction of the steel-ball sphere.

The loops include both endpoints. Longitude K duplicates the seam position at longitude 0 with a different U coordinate. Pole rows also contain repeated positions. The normal is obtained by normalizing the local position. The ball uses r = 0.28, L = 18 and K = 28.

5.4.1 Sphere Indices and Ball Vertices

$$a = i(K + 1) + j \; ; \; b = a + K + 1 \tag{5.9}$$

$$\text{triangle 1} = (a,b,a+1) \; ; \; \text{triangle 2} = (a+1,b,b+1) \tag{5.10}$$

Table 5.6: Selected exact ball-sphere samples from the indexed grid.

i, j	**Latitude**	**Longitude**	**Position**
0, 0	-90°	0°	(0, 0, -0.280000)
3, 0	-60°	0°	(0.140000, 0, -0.242487)
6, 0	-30°	0°	(0.242487, 0, -0.140000)
9, 0	0°	0°	(0.280000, 0, 0)
9, 7	0°	90°	(0, 0.280000, 0)
9, 14	0°	180°	(-0.280000, 0, 0)
12, 0	30°	0°	(0.242487, 0, 0.140000)
18, 0	90°	0°	(0, 0, 0.280000)

$$V = (L+1)(K+1) = 19 \times 29 = 551 \tag{5.11}$$

$$I = 6LK = 3024 \; ; \; T_{\text{submitted}} = 2LK = 1008 \tag{5.12}$$

The poles contribute 56 zero-area triangles, leaving 952 nondegenerate ball triangles for ray tracing. RayScene::addMesh rejects triangles whose squared cross-product length is below 10^{-16}.

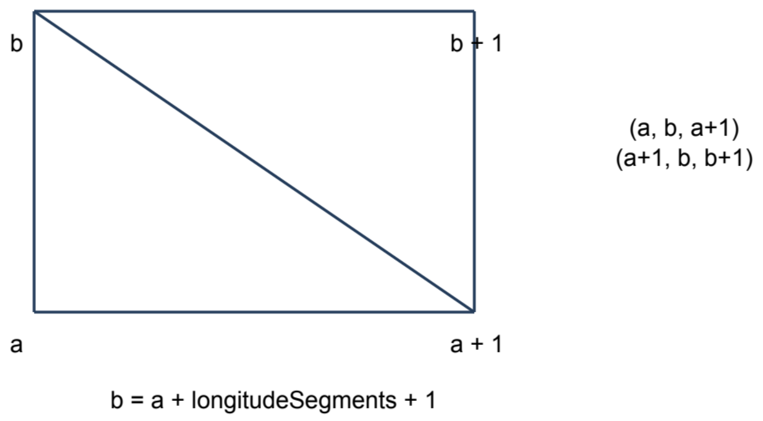


Figure 5.5: Two indexed triangles from one sphere grid cell.

5.5 Flipper Body and Cap Geometry

Source: Mesh.cpp - createFlipper(). The cuboid body has dimensions (1.48,0.22,0.16). Its half dimensions are (0.74,0.11,0.08), producing the same eight-corner pattern and 36 expanded body records as a cuboid.

$$r_{cap} = \sqrt{(0.11^2 + 0.08^2)} = 0.136015 \tag{5.13}$$

The source adds two sampled upper hemispherical patches centered at X = ±0.74. For cap sign $s = \pm 1$, φ ranges from 0 to $\pi/2$ and θ ranges from 0 to 2π:

$$p = (x_{cap} + s\, r_{cap} \cos\varphi \cos\theta,\ r_{cap} \cos\varphi \sin\theta,\ r_{cap} \sin\varphi) \tag{5.14}$$

$$N = (s \cos\varphi \cos\theta,\ \cos\varphi \sin\theta,\ \sin\varphi) \tag{5.15}$$

Each cap uses 12 latitude intervals and 12 longitude sectors, with six expanded records per cell. The total is 36 + 2 x 12 x 12 x 6 = 1764 records and 588 submitted triangles. These are upper patches in Z, not complete end hemispheres aligned with the X axis. The collision representation is a separate 2D capsule around the pivot-to-tip segment.

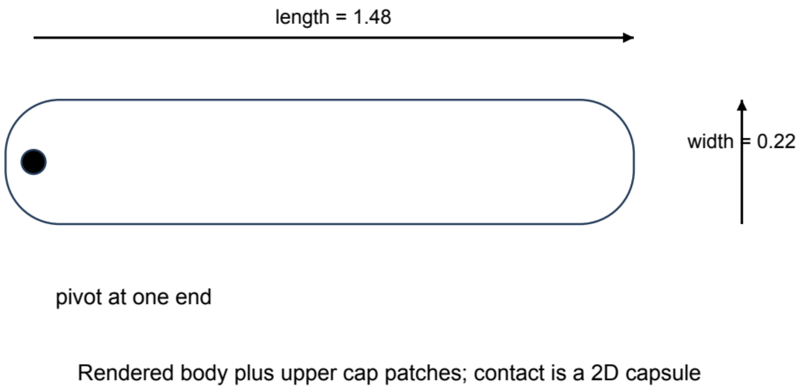

length = 1.48
width = 0.22
pivot at one end
Rendered body plus upper cap patches; contact is a 2D capsule

Figure 5.6: Flipper dimensions and the relationship between rendered geometry and contact geometry.

All 1764 records, their normals and the implicit triangle sequence are available in flipper_vertices.csv and flipper_triangles.csv. This keeps the exact generated data inspectable without printing thousands of repetitive rows in the main text.

5.6 Torus Ring

Source: Mesh.cpp - createRing(). A ring has major radius R and tube radius r. The two angles locate the tube center around the ring and a point around the tube cross-section:

$$\alpha_i = 2\pi i/N \; ; \; \beta_j = 2\pi j/M \tag{5.16}$$

$$p = ((R+r\cos\beta)\cos\alpha, (R+r\cos\beta)\sin\alpha, r\sin\beta) \tag{5.17}$$

$$N = (\cos\alpha\cos\beta, \sin\alpha\cos\beta, \sin\beta) \tag{5.18}$$

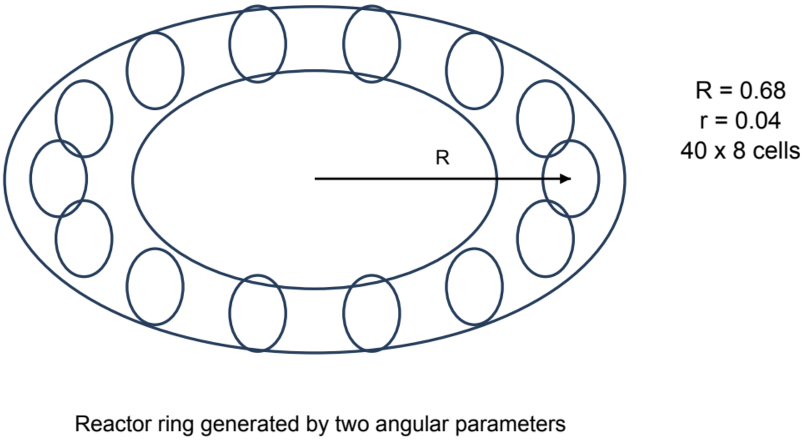


Figure 5.7: Torus geometry used by the reactor insert and bumper skirts.

Default sampling is N = 40 around the ring and M = 8 around the tube. Endpoint rows duplicate both seams. The ring stores (N+1)(M+1) = 369 vertices and 6NM = 1920 index entries, forming 640 triangles. The cell indexing follows the same two-triangle grid rule as the sphere.

Table 5.7: Reactor-ring samples with R = 0.68 and r = 0.04.

α	**β**	**Position**
0°	0°	(0.720000, 0, 0)
0°	90°	(0.680000, 0, 0.040000)
0°	180°	(0.640000, 0, 0)
90°	0°	(0, 0.720000, 0)

5.7 Helical Spring Factory

Source: Mesh.cpp - createSpring(). The factory is present in the source but is not called by the current scene. It therefore describes available procedural geometry rather than a visible production object.

$$c(t) = (r \cos(2\pi kt), r \sin(2\pi kt), -h/2 + ht) \quad (5.19)$$

The code samples $t_0 = i/(S-1)$ and $t_1 = (i+1)/(S-1)$, computes a segment tangent $\tau = c(t_1)-c(t_0)$, and builds a small eight-sided tube of radius 0.1r. A local frame uses $n = \text{normalize}(\tau \times Z)$ and $b = \text{normalize}(n \times \tau)$. The tube offset is $0.1r(\cos \beta\, b + \sin \beta\, n)$.

$$V_{spring} = 6 \times 8 \times S = 48S \; ; \; T = 16S \quad (5.20)$$

The loop runs $i < S$ rather than $i < S-1$, so the final endpoint reaches $t = S/(S-1)$, slightly beyond 1. The report records that implementation detail instead of replacing it with an idealized formula. A corrected future factory could use S-1 longitudinal intervals.

5.8 Text and Television Screen

createTextMesh uses a five-column, seven-row bitmap font. Each active pixel becomes one four-vertex quad in XZ, facing -Y, with two indexed triangles. Pixel size is textHeight/7; regular glyph advance is six pixels and a space advances four. The final X coordinates are shifted by half the accumulated text width.

Table 5.8: Television-screen quad from createScreenMesh(2.98,1.77).

ID	Position	Normal	UV
0	(-1.49, 0, -0.885)	(0,-1,0)	(0,0)
1	(1.49, 0, -0.885)	(0,-1,0)	(1,0)
2	(1.49, 0, 0.885)	(0,-1,0)	(1,1)
3	(-1.49, 0, 0.885)	(0,-1,0)	(0,1)

The screen index list is (0,1,2,0,2,3). Its direct UV layout avoids rotating the television artwork through an unsuitable cuboid face mapping.

5.9 Beanbags and Custom Rails

Source: GameHub.cpp - beanbagMesh(). The beanbag starts from spherical parameters $a = \pi i/24$ and $b = 2\pi j/40$. Let $x = \sin a \cos b$, $y = \sin a \sin b$, $z = \cos a$. The fold factor adds mild angular variation:

$$F = 1 + 0.035 \sin(11b + 7z) \sin^2 a \qquad (5.21)$$

$$p = (0.94xF, 1.12yF, \max(-0.4, 0.68z)) \qquad (5.22)$$

For $z > 0$, a Gaussian-shaped seat lowers the top and shifts it slightly toward +Y. The depression is $0.34 \exp[-x^2/0.45 - (y+0.25)^2/0.25]$. The positive hemisphere additionally receives 0.25z in Y. This produces an upholstered chair form rather than a perfect sphere. Normals accumulate negated face cross products and are normalized after all triangles are processed.

The beanbag stores 25 x 41 = 1025 vertices and 24 x 40 x 6 = 5760 indices. Nonuniform object scale and a Z-axis rotation then distinguish the two chairs.

5.10 Rails Derived from Collision Segments

A visible rail is built directly from its physical endpoints A and B. Their Z coordinates use the board-height function plus 0.18. The side vector is the XY perpendicular to B-A, normalized and multiplied by rail radius. Each endpoint receives ±side and ±(0,0,0.18), giving eight corners. Six faces each store four vertices and six indices: 24 records and 36 indices per rail.

$$\text{side} = r_{\text{rail}} \, \text{normalize}(-(B-A)_y, (B-A)_x, 0) \qquad (5.23)$$

Using the same XY endpoints for drawing and physics prevents guide geometry from drifting away from the collision path. Small triangular slingshot artwork is decorative and has no independent collider.

Chapter 6

Scene Construction and Reusable Meshes

Reusable factories create the machine and room components, while each GameObject owns its mesh and transform. The design reuses mathematical construction functions, not a global shared GPU primitive instance for every cuboid. Mesh objects are movable and noncopyable, preventing accidental duplication of OpenGL resource ownership.

Table 6.1: Principal machine components in simulation coordinates.

Object	Center or reference	Size or geometry
Cabinet	(0,0,0.22)	7.8 x 13.3 x 0.4 cuboid
Inclined deck	(0,0,1.25)	7.5 x (12.4/cos8°) x 0.24
Playfield	(0,0,1.4)	7 x (12/cos8°) plane
Chrome legs	(±3.5,±5.7,0.25)	0.15 x 0.15 x 0.5
Side rails	(±3.61,0,1.68)	0.22 x (12.25/cos8°) x 0.56
Backbox	(0,6.6,3.25)	7.8 x 0.65 x 2.5
Backglass	(0,6.23,3.35)	7.2 x 0.07 x 1.95
Glass cover	(0,0,2.04)	7.25 x (12/cos8°) x 0.02
Left flipper pivot	(-1.75,-4.6,z(-4.6)+0.12)	1.48 x 0.22 x 0.16 body
Right flipper pivot	(1.75,-4.6,z(-4.6)+0.12)	Same body dimensions
Ready ball center	(2.95,-4.6,z(-4.6)+0.28)	Sphere radius 0.28

Bumper body heights, target heights and flipper pivot heights are refreshed from calculateZ during physics updates. Decorative rings, lamps, lettering and rail meshes add visible detail without changing the physical game rules.

6.1 GPU Representation

Table 6.2: Vertex attributes and GPU bindings.

Location	Attribute	Components	Role
0	position	3 floats	Local surface point
1	normal	3 floats	Surface orientation
2	color	3 floats	Per-vertex color
3	texCoord	2 floats	Texture sampling coordinate

The VBO stores Vertex records, the VAO records their attribute layout, and the EBO stores indices when present. Attribute strides use sizeof(Vertex); normal, color and texture offsets use offsetof. The logical record has eleven floats; the precise byte size can depend on GLM alignment configuration.

```
if (!indices.empty())
    glDrawElements(GL_TRIANGLES, indices.size(),
                   GL_UNSIGNED_INT, 0);
else
    glDrawArrays(GL_TRIANGLES, 0, vertices.size());
```

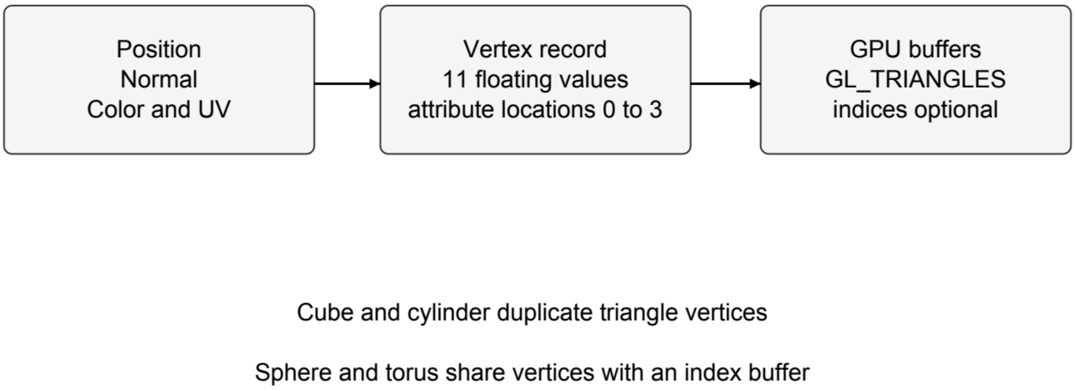

Position
Normal
Color and UV
Vertex record
11 floating values
attribute locations 0 to 3
GPU buffers
GL_TRIANGLES
indices optional
Cube and cylinder duplicate triangle vertices
Sphere and torus share vertices with an index buffer

Figure 6.1: Vertex attributes, buffer storage and the two drawing paths.

setupMesh returns early if GLAD function pointers are unavailable. The CPU vertices and textures remain valid, allowing the regression suites and geometry export to construct scenes without a window. TextureData holds an immutable CPU pixel copy shared by ray workers.

Chapter 7

Pinball Hierarchy and Flipper Kinematics

7.1 Flipper Dimensions and Stroke

Each flipper rotates around a fixed pivot in the XY board plane. The mesh is centered in its own frame, so a translation of ±length/2 is applied after the pivot rotation. The left mesh extends along +X; the right mesh extends along -X.

$$M_L = T(p_L)\ R_z(\theta_L)\ T(\ell/2,0,0)\ S \tag{7.1}$$

$$M_R = T(p_R)\ R_z(\theta_R)\ T(-\ell/2,0,0)\ S \tag{7.2}$$

Table 7.1: Flipper state and movement parameters.

Parameter	Left	Right
Pivot X,Y	(-1.75,-4.6)	(1.75,-4.6)
Rest angle	-28°	+28°
Active angle	+20°	-20°
Angular speed magnitude	720°/s	720°/s
Stroke magnitude	48°	48°
Full stroke time	48/720 ≈ 0.066667 s	Same

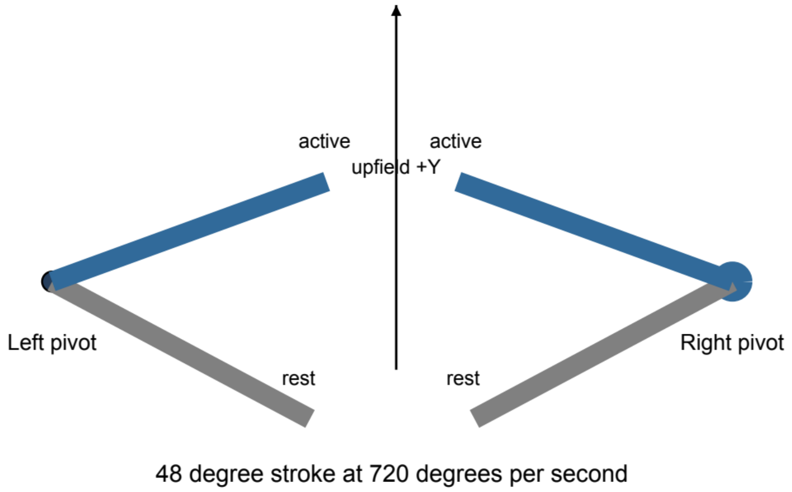

active
active
upfield +Y
Left pivot
Right pivot
rest
rest
48 degree stroke at 720 degrees per second

Figure 7.1: Opposing flippers at their rest and active angles.

7.1.1 Tip Direction

The collision direction is (cosθ,sinθ) for the left flipper and (-cosθ,-sinθ) for the right. Rotating this direction and adding pivot + length x direction gives the tip. The render and collider use the same angle and pivot.

7.2 Parent and Child Transform Chains

The complete pinball machine receives the lounge placement. Its playfield, ball, flippers, targets and decorations keep their own authoritative machine-space transforms. Room objects bypass this placement. The scene hierarchy is therefore a composition of transforms rather than a requirement for a single scene-graph class.

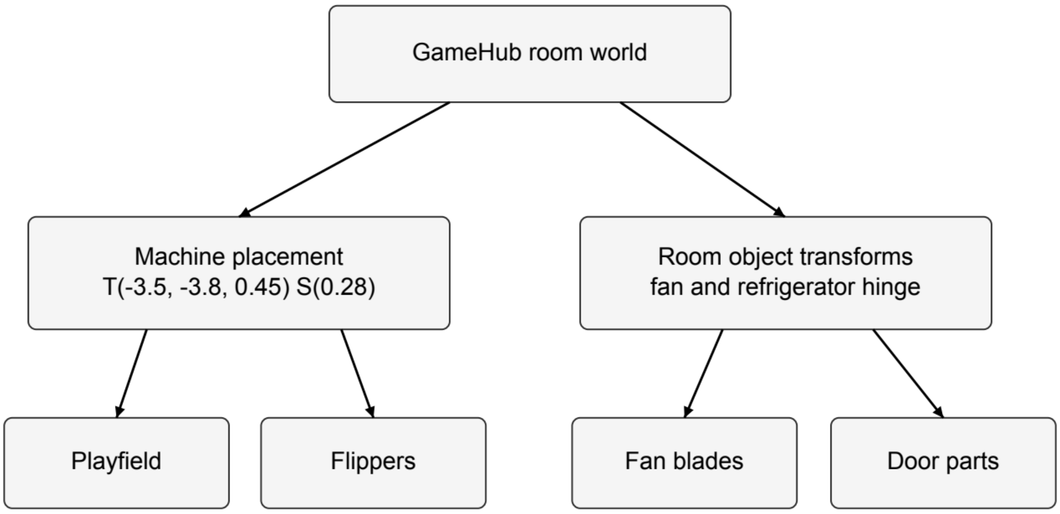

GameHub room world
Machine placement
T(-3.5, -3.8, 0.45) S(0.28)
Room object transforms
fan and refrigerator hinge
Playfield
Flippers
Fan blades
Door parts

Figure 7.2: Machine placement and the independent animated room assemblies.

7.3 Fan and Door Rigid Assemblies

$$\mathrm{bladeCenter}_i = c_{fan} + 0.77(\cos(\theta+90°i), \sin(\theta+90°i), 0) \tag{7.3}$$

$$\mathrm{doorPartWorld} = h_{door} + R_z(-\theta_{door})\ \mathrm{offset}_{part} \tag{7.4}$$

All four fan blades receive a common fan angle plus quadrant offsets. Each refrigerator frame part stores its offset from the hinge. The glass door, frame and handle use the same hinge rotation, preserving their rigid relationships while opening or closing.

The hinge is (-5.155,5.50,1.60), and the closed door center is offset by (0.495,0,0). At 100° open, the center is approximately (-5.240956,5.012520,1.600000). These are room coordinates; they do not use the machine scale.

Chapter 8

Ball Dynamics and Gameplay

8.1 Launch States and Plunger

The application has three ball states: READY, SHOOTER and IN_PLAY. READY keeps the ball motionless in the lane. A valid launch changes the state to SHOOTER and gives positive Y velocity. Crossing $X < 2.14$ and $Y > 4.0$ changes it to IN_PLAY. Scoring events are accepted only in IN_PLAY.

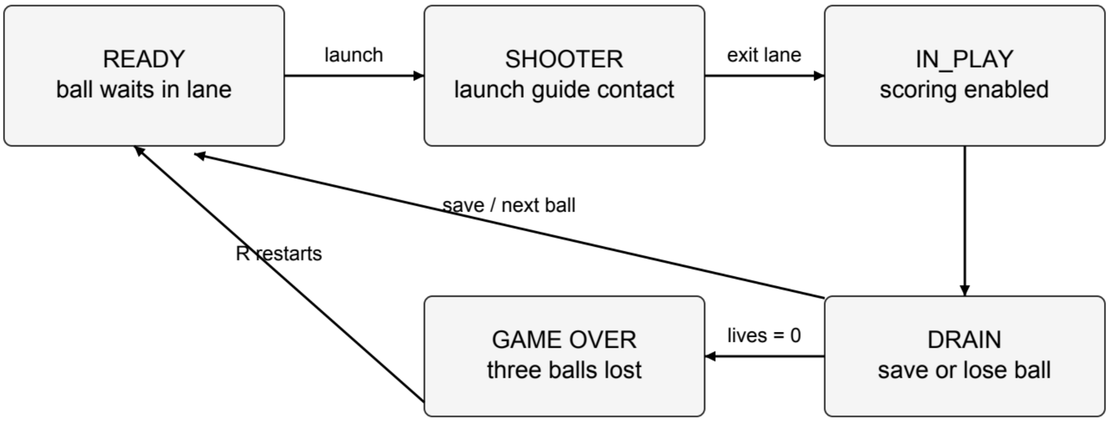

READY
ball waits in lane
launch
SHOOTER
launch guide contact
exit lane
IN_PLAY
scoring enabled
save / next ball
R restarts
GAME OVER
three balls lost
lives = 0
DRAIN
save or lose ball

Figure 8.1: Ball launch, play, drain and restart state transitions.

Space requests a launch speed of 18. Holding Enter increases normalized plunger compression at 1.6 per second, clamped at one. Releasing Enter launches with a speed calculated from compression c:

$$v_{launch} = 14 + 6c \; ; \; 0 \le c \le 1 \tag{8.1}$$

$$\text{plungerY} = -5.85 - 0.55c \tag{8.2}$$

launchFromLane clamps speed to 14-20 and ignores the request when the ball is already active or the game is over. A ball rolling backward into the shooter lane returns to READY. The player restarts a completed game explicitly with R.

8.2 Fixed Step Integration and Surface Constraint

PinballMachine::update accumulates time and performs physicsStep with $\Delta t = 1/240$ s. Input time is checked for finiteness and capped at 0.25 s per call. The visible frame may perform several physics steps, preserving the contact resolution at different rendering frame rates.

```
constexpr float step = 1.0f / 240.0f;
accumulator += std::min(dt, 0.25f);
while (accumulator + 1e-7f >= step) {
    physicsStep(step);
    accumulator = std::max(0.0f, accumulator-step);
}
```

The state velocity is stored in projected XY coordinates. For a board angle α, the implementation applies the downhill Y acceleration:

$$v_y \leftarrow v_y - 9.81 \sin \alpha \cos \alpha \, \Delta t \tag{8.3}$$

$$a_y = -9.81 \sin 8° \cos 8° \approx -1.352001 \tag{8.4}$$

Ball::update forces vertical velocity to zero, applies the time-adjusted damping factor, caps speed at 22 and advances position. After contacts, seatBall restores the center height from the board surface.

$$v \leftarrow v \, (0.999)^{60\Delta t} \; ; \; p \leftarrow p + v\Delta t \tag{8.5}$$

$$z_{center} = 1.4 + y \tan 8° + 0.28 \; ; \; v_z = 0 \tag{8.6}$$

This is a surface-constrained game model. It does not simulate free-flight jumps, ball spin or exact rolling inertia. The gravity projection and vertical seating are implementation choices that support stable gameplay, rather than a full rigid-body simulation.

8.3 Scoring Events and Contact Cooldowns

Table 8.1: Scoring rules before applying the current multiplier.

Event	Base score	Added bonus	Guard
Left/right main bumper	100	10	0.18 s hit timer and incoming contact
Upper-middle bumper	150	10	Same bumper guard
Small side bumper	50	10	Same bumper guard
Drop target	100	50	Target must be standing
Slingshot	10	0	0.25 s cooldown per side
Left orbit	250	100	Upward traversal; 1.5 s cooldown
Four-target bank	1000	0	All four targets down
Flipper or ordinary wall	0	0	No scoring event

$$\text{scoreIncrement} = \text{basePoints x currentMultiplier} \tag{8.7}$$

Bumper scoring requires an approaching ball with incoming normal velocity less than -0.05 and no active hit timer. Target scoring changes down to true and then ignores further collisions until the bank resets. A slingshot applies its impulse only on a valid impact and after its cooldown has expired.

The orbit must first be armed in the left side lane: $X < -1.9$, $-1.8 < Y < 2.4$ and positive Y speed above 1. It completes only after reaching $Y > 4.35$, $X < -0.8$ and upward velocity. Falling below $Y = -2.2$ disarms it. This prevents the region from awarding points solely because the ball remains near the top.

Score is capped at 99,999,999. The session best updates whenever total score grows, remains when R resets the game, and is not persisted to disk after application exit.

8.4 Bonus Multipliers Ball Saves and Lives

Completing the four-target bank gives 1000 points using the multiplier that existed before completion. It then raises the multiplier by one, up to 5x, and schedules a bank reset after 1.25 seconds. Ordinary target cooldown values do not independently raise the whole bank; resetTargetBank controls the bank state.

$$m_{new} = \min(5, m_{old} + 1) \tag{8.8}$$

For the first bank at 1x, the four targets produce 400 and the bank award produces 1000, giving 1400. Their pending bonus totals 200. A subsequent unsaved drain collects 200 x 2 = 400, taking score to 1800 and resetting bonus and multiplier.

$$\text{drainBonus} = \text{pendingBonus} \times \text{currentMultiplier} \tag{8.9}$$

The first launch of each ball enables an eight-second ball save once. An early drain consumes the remaining save time and returns the same ball to READY without losing a life or clearing its bonus/multiplier. Relaunching that saved ball does not provide a second save. An unsaved drain decrements lives, resets the bank and bonus, and prepares the next ball. Exactly three lost balls produce game over.

Table 8.2: State changes for a save and an ordinary lost ball.

State value	Saved drain	Unsaved drain
Lives and ball number	Preserved	Lives -1; advance ball if lives remain
Pending bonus and multiplier	Preserved	Collect bonus; reset to 0 and 1x
Target bank	Preserved	Reset all four targets
Ball save	Consumed	New ball receives one save on launch
Ball state	READY	READY; gameOver if no lives

Regression checks explicitly verify the 1400-point bank, the multiplied drain bonus, one-time drain accounting, three-ball game over and session-best preservation.

Chapter 9

Lighting Theory and Implementation

9.1 Phong Lighting Vectors

The principal shader uses the Phong reflection model. At a surface point p, N is a normalized surface normal, L points toward the light, and V points toward the camera. The reflected light direction R is computed from the incoming direction -L:

$$L = \text{normalize}(\text{lightPosition} - p)\ ;\ V = \text{normalize}(\text{cameraPosition} - p) \tag{9.1}$$

$$R = \text{reflect}(-L,N) = 2(N \cdot L)N - L \tag{9.2}$$

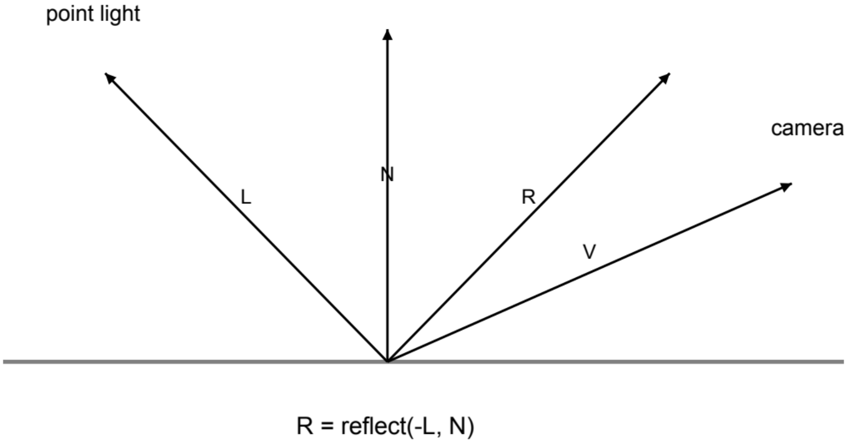


Figure 9.1: Vectors used by the project's Phong diffuse and specular calculation.

9.2 Ambient Diffuse and Specular Terms

$$f_d = \max(N \cdot L,0)\ ;\ f_s = [\max(V \cdot R,0)]^{\max(s,1)} \tag{9.3}$$

Ambient approximates background illumination. Diffuse changes with the surface-light angle. Specular gives view-dependent highlights, controlled by shininess s. This implementation uses R, not the Blinn-Phong halfway vector. The naming of the shading path and the chosen lighting model are separate concepts.

9.3 Implemented Color Contributions

Let a be albedo, formed from objectColor, vertex Color and a sampled texture when enabled. For one point light with intensity I and attenuation A, the raster Phong shader computes:

$$C_{point} = A\ I\ a \odot (L_a + L_d f_d + L_s f_s) \tag{9.4}$$

Here ⊙ denotes component-wise RGB multiplication. In the raster shader, albedo multiplies the specular contribution as well as ambient and diffuse. The ray tracer uses albedo on ambient and diffuse but leaves the direct specular color untinted; its output therefore need not match raster output pixel for pixel.

9.4 Distance Attenuation

$$A(d) = 1 / (k_c + k_l d + k_q d^2) \tag{9.5}$$

9.5 Spotlight Cone Factor

$$q = L \cdot \text{normalize}(-\text{spotDirection}) \tag{9.6}$$

$$c = \text{clamp}((q - \cos\theta_{outer}) / \max(\cos\theta_{inner} - \cos\theta_{outer}, 0.0001), 0, 1) \tag{9.7}$$

$$c_{shaped} = c^{\max(0.05\ \text{exponent}, 0.2)} \tag{9.8}$$

The active table spotlight uses inner 30°, outer 45° and exponent 20, so the shaping power is one. Phong/Gouraud apply the cone to diffuse and specular; spotlight ambient remains outside the cone factor.

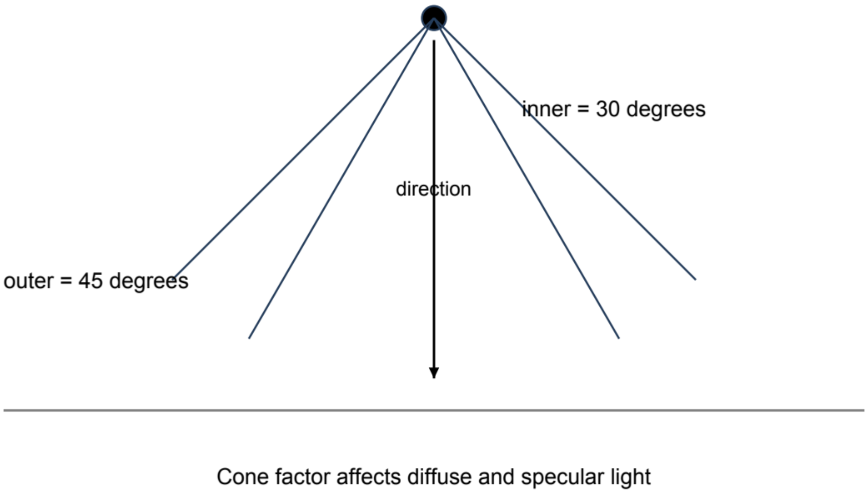


Figure 9.2: Inner and outer cone limits for the table spotlight.

9.6 Effective Light Rig

GameHub supplies seven point lights to both the room and the placed machine. Although PinballMachine creates a separate three-light configuration, GameHub::render calls machine.render(shader,false), so the integrated application uses the GameHub rig instead.

Table 9.1: Effective room point-light configuration.

ID	Room position	RGB diffuse	Intensity
0	(1.0,2.8,4.7)	(0.79,0.90,1.00)	3.20
1	(-3.2,5.2,4.75)	(0.88,0.95,1.00)	2.50
2	(2.0,5.9,3.9)	(0.11,0.36,1.00)	1.50
3	(4.9,5.95,1.1)	(0.24,0.09,1.00)	1.15
4	(-4.66,5.25,1.9)	(0.50,0.74,1.00)	0.45
5	(-3.5,-3.8,1.85)	(0.80,0.88,1.00)	1.10
6	(5.2,2.5,3.9)	(0.52,0.70,1.00)	0.25 night; 2.20 day

Point lights use constant 1, linear 0.045 and quadratic 0.025 attenuation. Ambient color is 0.045 times the diffuse color; specular is white. Switching room lights off disables lights 0-3. Lights 4-6 remain enabled, preserving refrigerator, machine and window contributions.

Table 9.2: Active table spotlight values.

Property	Value
Position and direction	(-3.5,-3.8,3.55); (0,0,-1)
Inner / outer cutoffs	30° / 45°
Intensity and exponent	2.0 and 20
Attenuation	kc = 1; kl = 0.05; kq = 0.02
Enable policy	Enabled when table camera is active or room lights are on

Room fixtures also have emissive materials. In raster mode their emission brightens their own surfaces; point/spot lights approximate their illumination of nearby objects. The ray tracer can see emissive fixtures in reflected and transmitted paths, but does not sample arbitrary emissive surfaces as diffuse area lights.

9.7 Day Night and Room Light Control

Table 9.3: Scene ambient RGB under the four room-light conditions.

Condition	Ambient RGB
Lights on, day	(0.32,0.37,0.42)
Lights on, night	(0.22,0.27,0.34)
Lights off, day	(0.12,0.16,0.22)
Lights off, night	(0.018,0.026,0.050)

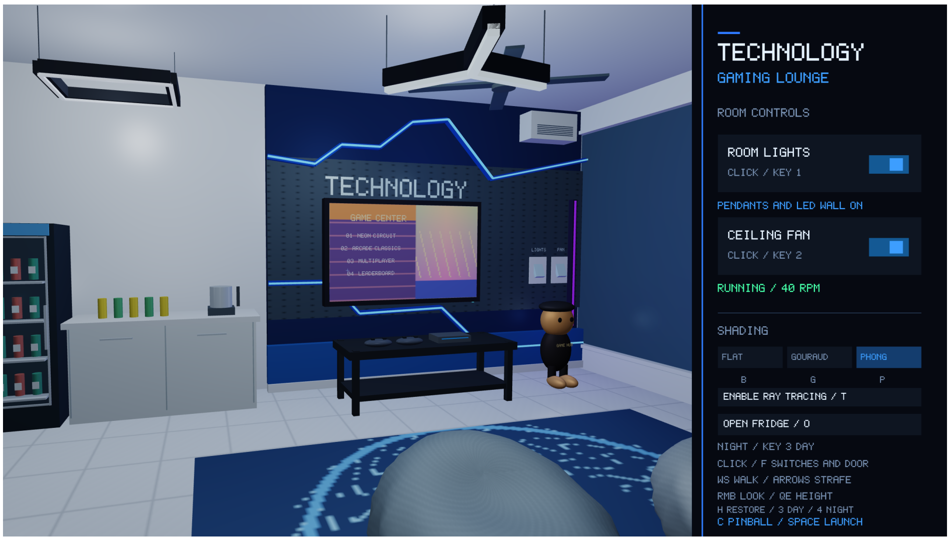

TECHNOLOGY
GAMING LOUNGE
ROOM CONTROLS
ROOM LIGHTS
CLICK / KEY 1
PENDANTS AND LED WALL ON
CEILING FAN
CLICK / KEY 2
RUNNING / 40 RPM
SHADING
FLAT
GOURAUD
PHONG
B
G
P
ENABLE RAY TRACING / T
OPEN FRIDGE / O
NIGHT / KEY 3 DAY
CLICK / F SWITCHES AND DOOR
WS WALK / ARROWS STRAFE
RMB LOOK / QE HEIGHT
H RESTORE / 3 DAY / 4 NIGHT
C PINBALL / SPACE LAUNCH
TECHNOLOGY

Figure 9.3: Room lighting enabled in the Phong raster view.

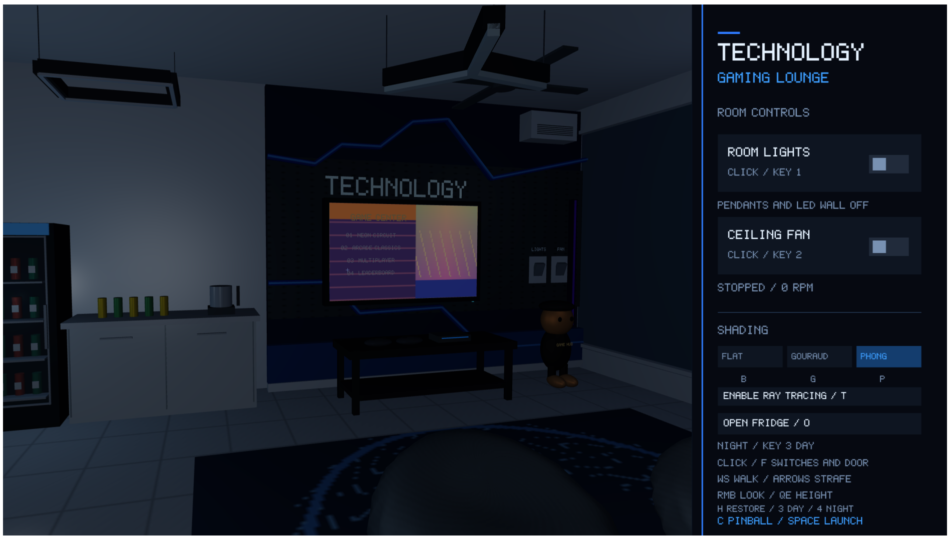

TECHNOLOGY
GAMING LOUNGE
ROOM CONTROLS
ROOM LIGHTS
CLICK / KEY 1
PENDANTS AND LED WALL OFF
CEILING FAN
CLICK / KEY 2
STOPPED / 0 RPM
SHADING
FLAT
GOURAUD
PHONG
B
G
P
ENABLE RAY TRACING / T
OPEN FRIDGE / O
NIGHT / KEY 3 DAY
CLICK / F SWITCHES AND DOOR
WS WALK / ARROWS STRAFE
RMB LOOK / QE HEIGHT
H RESTORE / 3 DAY / 4 NIGHT
C PINBALL / SPACE LAUNCH
TECHNOLOGY

Figure 9.4: Actual room capture with room lights disabled.

Keys 1, 3 and 4 select room-light power, day mode and night mode. Light-surface emission follows the room switch; fan power is independent. The dark-room image still receives the explicitly retained local lights and ambient policy.

Chapter 10

Materials and Shading

10.1 Material Parameters

A GameObject stores base RGB color, alpha, emissiveStrength and shininess. It also carries reflectivity and refractiveIndex for ray tracing. These scalar and vector properties specialize reusable geometry without changing vertex positions.

Table 10.1: Representative material values used by the source.

Object	Shininess	Emission	Alpha / ray property
Steel ball	96	0.12	Opaque; reflection 0.72
Plunger	96	0	Opaque; reflection 0.40
Bumper body	48	0.15; 1.0 on hit	Opaque
Drop target	24	0.20; 0.80 down	Opaque
Flipper	32	0.10	Opaque
Playfield	40	0.25	Opaque; artwork texture
Porcelain floor	120	0	Opaque; reflection 0.18
Beanbag fabric	5	0	Opaque; upholstery texture
Window glass	128	0	Alpha 0.12; IOR 1.5
Refrigerator glass	100	0	Alpha 0.14; IOR 1.5
Machine glass cover	32	0	Alpha 0.035; IOR 1.5
Kettle	100	0	Reflection 0.50

Raster opacity uses alpha blending. RayScene maps alpha to transmission = 1-alpha for glass. Reflectivity mixes recursive reflection with local surface lighting for opaque materials. Glass uses Fresnel reflectance instead of a fixed mirror weight.

10.2 Lighting Model and Shading Method

The lighting model defines the color calculation. The shading method determines where the calculation occurs and which quantities are interpolated across the triangle. The executable selects a separate shader program for each named mode.

10.2.1 Flat Mode in This Implementation

flat.vert passes ordinary interpolated position, normal, color and UV outputs. flat.frag evaluates ambient and diffuse illumination per fragment, omitting the specular term. It has no flat-qualified varyings and does not derive a constant geometric face normal. Cuboid faces still look planar because their mesh records use constant face normals, but smooth meshes remain smoothly interpolated.

10.2.2 Gouraud Shading

gouraud.vert evaluates ambient, diffuse, specular and emission at each vertex and emits LightingColor. The fragment shader interpolates that color, multiplies it by vertex/texture color and performs tone mapping. A narrow highlight can be missed when it falls between coarse mesh vertices.

10.2.3 Phong Shading

phong.vert emits world position and transformed normal. phong.frag renormalizes the interpolated normal and evaluates lighting for each fragment. This generally resolves specular detail more accurately on the steel ball, cylinders and polished surfaces.

$$\text{radiance} = 0.75(\text{result} + \text{albedo} \times \text{emission}) \tag{10.1}$$

$$\text{displayRGB} = \text{radiance} / (1 + \text{radiance}) \tag{10.2}$$

Both equations act component-wise. Gouraud performs the equivalent exposure and tone mapping after vertex-light interpolation and texture multiplication. The code does not apply an explicit gamma conversion.

10.3 Visual Comparison of Shading Modes

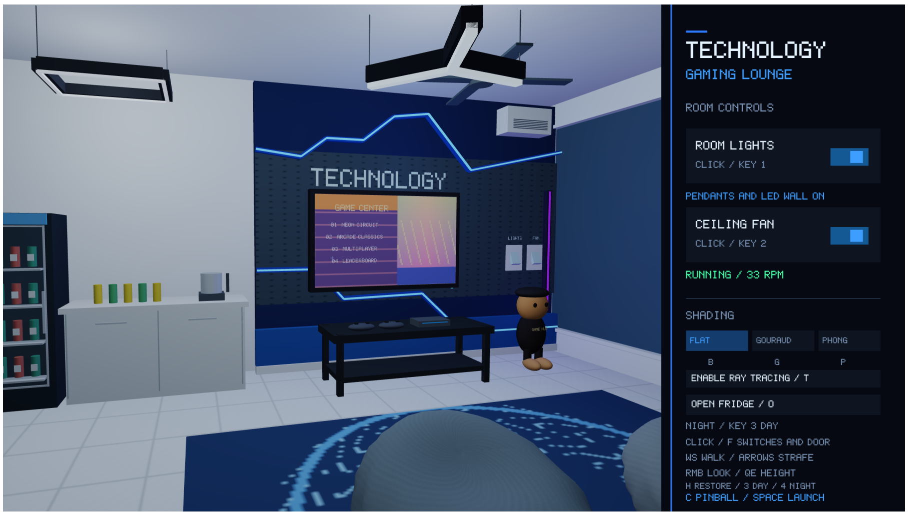

TECHNOLOGY
GAMING LOUNGE
ROOM CONTROLS
ROOM LIGHTS
CLICK / KEY 1
PENDANTS AND LED WALL ON
CEILING FAN
CLICK / KEY 2
RUNNING / 33 RPM
SHADING
FLAT
GOURAUD
PHONG
B
G
P
ENABLE RAY TRACING / T
OPEN FRIDGE / O
NIGHT / KEY 3 DAY
CLICK / F SWITCHES AND DOOR
WS WALK / ARROWS STRAFE
RMB LOOK / QE HEIGHT
H RESTORE / 3 DAY / 4 NIGHT
C PINBALL / SPACE LAUNCH
TECHNOLOGY
GAME CENTER

Figure 10.1: Mode labeled Flat with ambient and diffuse lighting.

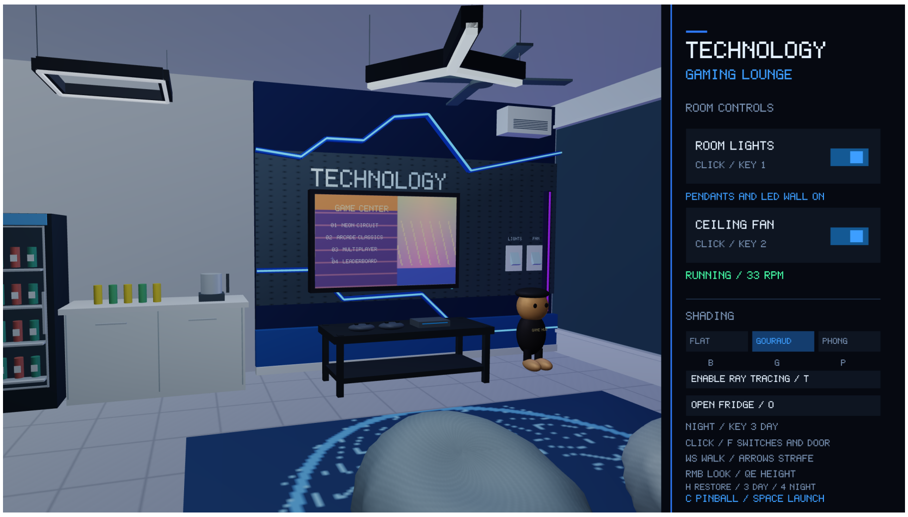

TECHNOLOGY
GAMING LOUNGE
ROOM CONTROLS
ROOM LIGHTS
CLICK / KEY 1
PENDANTS AND LED WALL ON
CEILING FAN
CLICK / KEY 2
RUNNING / 33 RPM
SHADING
FLAT
GOURAUD
PHONG
B
G
P
ENABLE RAY TRACING / T
OPEN FRIDGE / O
NIGHT / KEY 3 DAY
CLICK / F SWITCHES AND DOOR
WS WALK / ARROWS STRAFE
RMB LOOK / QE HEIGHT
H RESTORE / 3 DAY / 4 NIGHT
C PINBALL / SPACE LAUNCH
TECHNOLOGY
GAME CENTER

Figure 10.2: Gouraud mode with lighting evaluated at vertices.

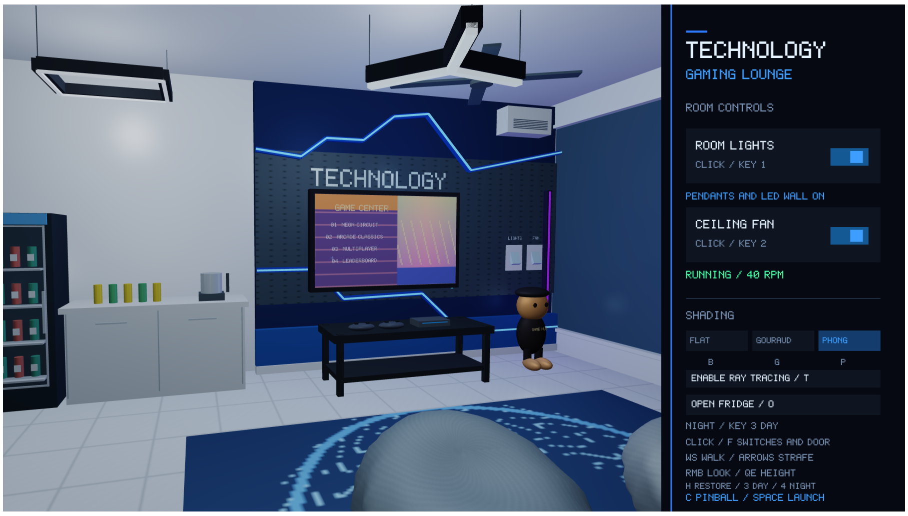

TECHNOLOGY
GAMING LOUNGE
ROOM CONTROLS
ROOM LIGHTS
CLICK / KEY 1
PENDANTS AND LED WALL ON
CEILING FAN
CLICK / KEY 2
RUNNING / 40 RPM
SHADING
FLAT
GOURAUD
PHONG
B
G
P
ENABLE RAY TRACING / T
OPEN FRIDGE / O
NIGHT / KEY 3 DAY
CLICK / F SWITCHES AND DOOR
WS WALK / ARROWS STRAFE
RMB LOOK / QE HEIGHT
H RESTORE / 3 DAY / 4 NIGHT
C PINBALL / SPACE LAUNCH
TECHNOLOGY
GAME CENTER

Figure 10.3: Phong mode with lighting evaluated per fragment.

Chapter 11

Normal Transformation

A normal must stay perpendicular to its transformed surface. Multiplying it by the model's linear part is generally incorrect under nonuniform scale. The vertex shaders apply the inverse transpose of the combined scene and object transform:

$$A = (M_{scene}M_{object})_{3x3}\ ;\ N_{matrix} = (A^{-1})^{T} \tag{11.1}$$

$$N_{world} = \mathrm{normalize}(N_{matrix}N_{local}) \tag{11.2}$$

```
Normal = mat3(transpose(inverse(sceneTransform * model)))
         * aNormal;
```

To see why, let tangent t and normal n satisfy $n \cdot t = 0$. After transforming the tangent to At, the candidate normal $A^{-T}n$ gives $(A^{-T}n) \cdot (At) = n \cdot t = 0$. The perpendicular relationship is preserved.

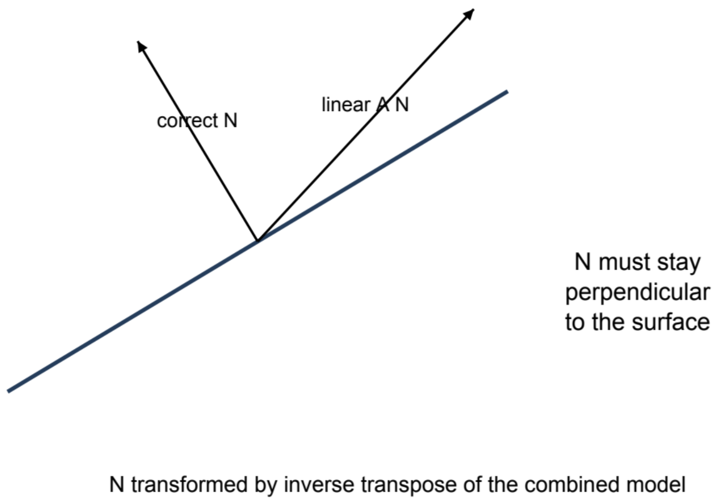

correct N
linear A N
N must stay perpendicular to the surface
N transformed by inverse transpose of the combined model

Figure 11.1: Correct and incorrect normal directions after a nonuniform transform.

RayScene::addMesh uses the same inverse-transpose rule for each world-space vertex normal and normalizes the result. It computes a geometric triangle normal for intersection-side classification, aligns that normal with the vertex-normal sum, and discards degenerate triangles. These are distinct roles for shading normals and geometric normals.

Chapter 12

Camera System

12.1 Room and Table Cameras

Camera stores position, worldUp, yaw, pitch, speed, mouse sensitivity and vertical field of view. It is a movable first-person-style camera rather than an orbit-around-target controller. Its Z-up forward vector is derived from yaw ψ and pitch ϕ:

$$\mathrm{front} = \mathrm{normalize}(\cos\psi\cos\phi, \sin\psi\cos\phi, \sin\phi) \tag{12.1}$$

$$\mathrm{right} = \mathrm{normalize}(\mathrm{front}\ \text{x}\ \mathrm{worldUp})\ ;\ \mathrm{up} = \mathrm{normalize}(\mathrm{right}\ \text{x}\ \mathrm{front}) \tag{12.2}$$

$$V = \mathrm{lookAt}(\mathrm{position}, \mathrm{position} + \mathrm{front}, \mathrm{up}) \tag{12.3}$$

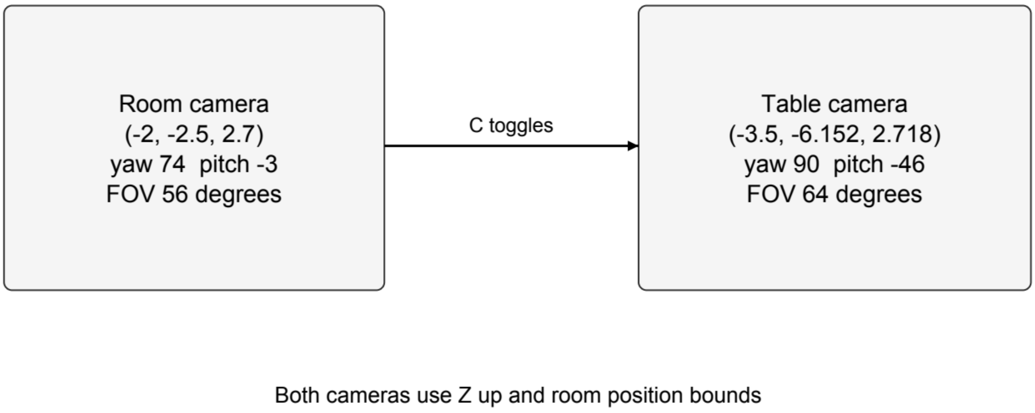

Room camera
(-2, -2.5, 2.7)
yaw 74 pitch -3
FOV 56 degrees
C toggles
Table camera
(-3.5, -6.152, 2.718)
yaw 90 pitch -46
FOV 64 degrees
Both cameras use Z up and room position bounds

Figure 12.1: Default room and table camera states after machine placement.

The room camera starts at (-2,-2.5,2.7), yaw 74°, pitch -3° and FOV 56°. The table camera originates in machine coordinates at (0,-8.4,8.1), yaw 90°, pitch -46° and FOV 64°. Applying the machine placement gives room position (-3.5,-6.152,2.718). Its room movement speed is then set to 1.6.

Camera position is clamped inside $X \in [-5.75,5.75]$, $Y \in [-6.75,6.75]$, $Z \in [0.25,5.35]$. This is containment within room bounds; it is not collision avoidance against each piece of furniture.

12.2 Perspective Projection

Both cameras use glm::perspective with near plane n = 0.1 and far plane f = 400. The aspect ratio is the width of the 3D viewport divided by framebuffer height. Since the console occupies the right side, the 3D viewport is approximately 73% of framebuffer width.

$$q = 1 / \tan(FOV_y/2) \; ; \; a = \text{viewportWidth} / \text{viewportHeight} \tag{12.4}$$

$$P = \begin{bmatrix} q/a & 0 & 0 & 0 \\ 0 & q & 0 & 0 \\ 0 & 0 & -(f+n)/(f-n) & -2fn/(f-n) \\ 0 & 0 & -1 & 0 \end{bmatrix} \tag{12.5}$$

12.3 Movement Zoom and Picking Rays

W/S move along front; the left/right arrows strafe along right; Q/E move along world Z. Movement distance is speed x frame time. Right-mouse movement changes yaw and pitch at 0.1° per pixel, with pitch clamped to ±89°. Scroll changes FOV within 25°-75°.

$$\text{clip} = (2x/\text{viewWidth} - 1, 1 - 2y/\text{height}, -1, 1) \tag{12.6}$$

For a scene click, inverse projection maps the clip point into eye space. The code then sets eye.z = -1 and eye.w = 0 and applies inverse view to obtain a normalized world ray. Window-to-framebuffer scaling handles high-resolution displays. The scene picker tests switches and the moving refrigerator door in their appropriate coordinate frames.

RayTracer::primaryRay uses the same camera basis and FOV. A regression test projects a generated ray through the raster matrices and checks that it returns the intended pixel coordinate.

Chapter 13

Animation and Time Based Motion

13.1 Ceiling Fan Acceleration and Coasting

The fan rotates around Z at a maximum angular speed of 240°/s, equivalent to 40 RPM. Turning it on ramps speed toward that target at $120°/s^2$. Turning it off reduces speed toward zero at $160°/s^2$, producing a visible coast before stopping.

$$\omega \leftarrow \omega + \mathrm{clamp}(\omega_{target} - \omega, -a\Delta t, a\Delta t) \tag{13.1}$$

$$\theta \leftarrow (\theta + \omega\Delta t) \bmod 360° \tag{13.2}$$

From rest, reaching 240°/s requires approximately two seconds. Coasting from full speed requires approximately 1.5 seconds. Four blade centers orbit FAN_CENTER = (2.3,4.1,5.07), with radius 0.77 and 90° angular spacing.

13.2 Hinged Refrigerator Door

$$\theta_{door} \leftarrow \theta_{door} + \mathrm{clamp}(\theta_{target} - \theta_{door}, -100\Delta t, 100\Delta t) \tag{13.3}$$

The door target is 100° when open and 0° when closed. A complete opening takes approximately one second. Toggling during motion changes the target immediately, so the same angle reverses direction continuously. All door parts rotate together using -θ about the hinge.

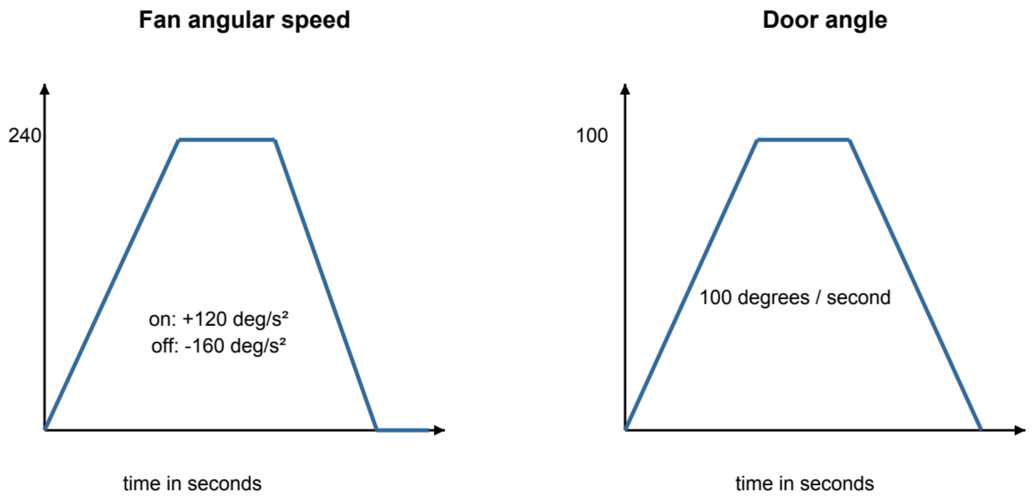


Figure 13.1: Idealized time profiles defined by the fan and door update rules.

13.3 Pinball Animation and Event Feedback

Flippers advance at 720°/s toward their powered or resting angles and clamp at the two stroke limits. Angular velocity is measured from the actual angle change divided by the physics step. This measured value supplies contact-surface velocity for ball impacts.

$$\Delta\theta = \mathrm{clamp}(\theta_{\mathrm{target}} - \theta, -720\Delta t, 720\Delta t) \tag{13.4}$$

A bumper hit changes its color to yellow for 0.18 seconds. Its associated jewel lamp also increases emission. A dropped target changes color, moves down by 0.18 in its model transform and rotates 70° about X. This target pose changes immediately when down becomes true; the 1.25-second bank timer controls the later reset rather than a continuous lowering animation.

The plunger visually follows its normalized compression. A waiting ball remains seated on the slope. Ball movement follows the same fixed-step position used by its rendered sphere and the ray snapshot. The model has no explicit spin angle derived from rolling distance.

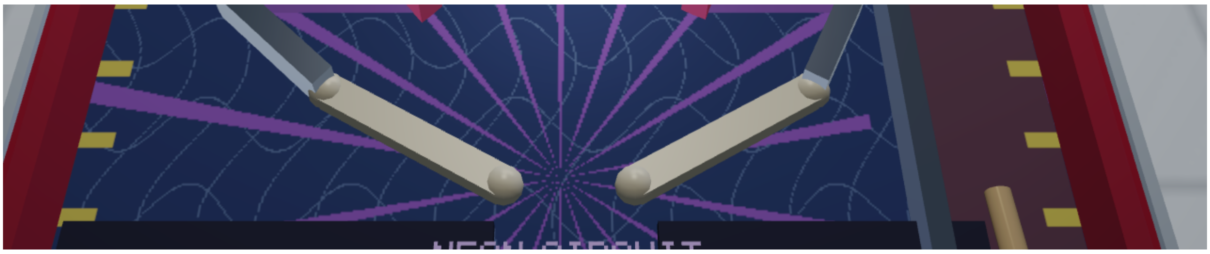


Figure 13.2: Flippers and lower-board artwork from the actual game capture.

Table 13.1: Time-dependent update paths.

System	Time handling	Visible result
Pinball machine	Fixed 1/240 s substeps	Consistent ball and contact updates
Fan / door	Frame dt capped at 0.25 s	Smooth speed and hinge movement
Bumper and event timers	Countdown during physics steps	Brief color and console feedback
Ray image	Asynchronous scene snapshot	Latest completed frame may lag animation

Chapter 14

Collision Detection and Contact Response

14.1 Ball and Circular Bumper

Bumper contact is checked in XY. For ball center P, bumper center C, ball radius r and bumper radius R, contact occurs when |P-C| < r+R. A normalized separation vector gives the contact normal; the code uses (0,1) when centers nearly coincide.

$$n = (P-C)/|P-C| \text{ ; overlap} = r+R-|P-C| \tag{14.1}$$

$$P \leftarrow P + \text{overlap } n \tag{14.2}$$

$$v' = v - (1+e)(v \cdot n)n \text{ ; only if } v \cdot n < 0 \tag{14.3}$$

Ball restitution is 0.72 for its bounce helper. A valid bumper hit adds an outward impulse of 8.5 for main bumpers or 6.5 for side bumpers, then activates the hit timer. Contact correction can occur even when a cooldown suppresses another score.

14.2 Ball and Rectangular Target

The closest point on a target rectangle is C + (clamp(Px-Cx,-halfW,halfW), clamp(Py-Cy,-halfD,halfD)). A ball touches it when distance to this point is less than ball radius. The target uses half-width 0.22 and half-depth 0.08; a coincident closest point falls back to normal (0,-1).

$$Q = C + \text{clamp}(P-C, -\text{halfSize}, \text{halfSize}) \tag{14.4}$$

A hit pushes the ball out, applies the bounce helper and sets down = true. Down targets skip further collision checks. The target is an axis-aligned XY contact rectangle even though its rendered dropped pose is rotated.

14.3 Ball and Finite Rail Segment

$$t = \mathrm{clamp}(((P-A) \cdot (B-A)) / |B-A|^2, 0, 1) \tag{14.5}$$

$$Q = A + t(B-A) \text{ ; contact if } |P-Q| < r_{ball} + r_{rail} \tag{14.6}$$

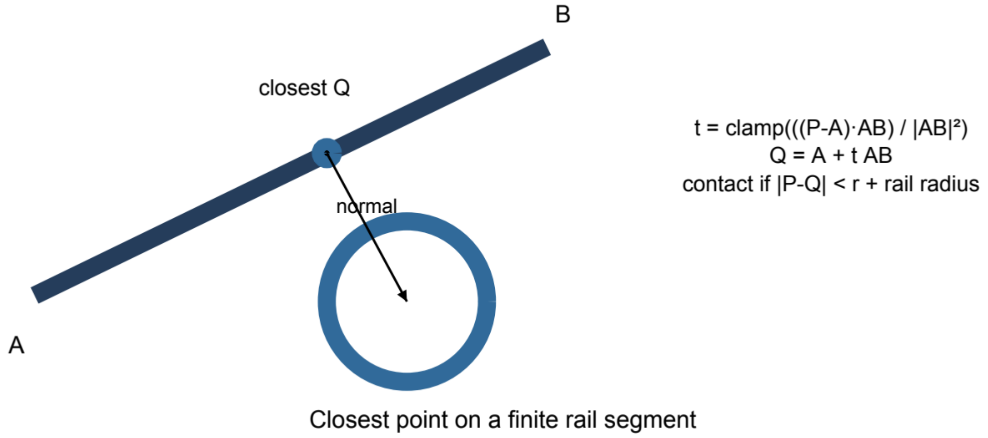

B
closest Q
normal
A
t = clamp(((P-A)·AB) / |AB|²)
Q = A + t AB
contact if |P-Q| < r + rail radius
Closest point on a finite rail segment

Figure 14.1: Closest-point contact between the ball and a finite rail segment.

A penetration correction adds a small 0.0001-unit separation. For incoming normal speed $v \cdot n < 0$, velocity changes by $-(1+\text{bounce})(v \cdot n)n$. Each rail has its own radius and restitution. Slingshot rails also supply an extra impulse after their scoring guard succeeds.

14.4 Moving Flipper Contact

$$v_{surface} = \omega \times r = (-\omega r_y, \omega r_x) \tag{14.7}$$

$$v_{relative} = v_{ball} - v_{surface} \tag{14.8}$$

The contact segment extends from the pivot along its current direction for length 1.48. Contact radius is ball radius plus half the flipper width. The code reflects incoming relative normal velocity with coefficient 1.6, corresponding to a 0.6 restitution factor in that response. Powered upward strokes can additionally enforce Y speed of at least 8 + 8t and add a side-dependent X kick of magnitude 2t.

14.5 Walls Drain and Numerical Safeguards

The side-wall center limits are ±(3.5-0.11-0.28) = ±3.11. The top limit is 6-0.11-0.28 = 5.61. X-wall impacts use restitution 0.7 and the upper wall uses 0.8. The lower edge remains open so a missed ball can drain.

$$\text{drain condition: ballY} < -6 - 0.28 = -6.28 \tag{14.9}$$

Collision response is discrete rather than continuous swept-volume collision. The fixed 240 Hz update reduces tunneling risk at the selected launch and flipper speeds. The main Ball update caps speed before position integration; the flipper response also reapplies the cap after its kick. Bumper impulses can briefly exceed the cap until the next step.

Table 14.1: Robustness measures in contact and interaction code.

Case	**Implemented handling**
Non-finite or invalid time	Ignore invalid dt; cap accumulated frame contribution
Zero-length rail	Skip if segment squared length is below 10^{-8}
Coincident circular centers	Use deterministic fallback contact normal
Repeated bumper contact	Separate contact resolution from hit-timer scoring
Ball in READY	Seat it without advancing normal gameplay
Camera leaving room	Clamp position to interior bounds
Near-parallel picking ray	Require origin inside the corresponding slab
Tiny ray triangle	Skip degenerate triangle during export

The project does not use the robotic reference report's OBB/SAT safety system for gameplay. Its actual pinball contact model uses circles, rectangles and finite capsules in XY. AABB slab tests appear in room picking and ray acceleration, which serve different purposes.

Chapter 15

Gaming Lounge and Ray Tracing

15.1 Lounge Layout

The room provides context for the playable machine and a larger scene for lighting, transparency and ray tracing. Its generated geometry occupies a 12 x 14 floor with height 5.6. The pinball machine is placed at (-3.5,-3.8,0.45), leaving the central rug and television area open.

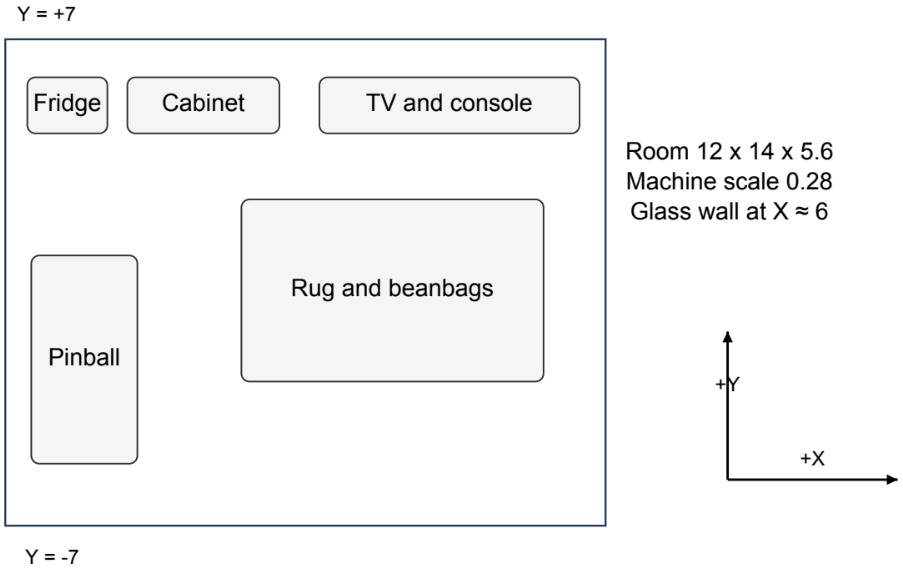

Y = +7
Fridge
Cabinet
TV and console
Room 12 x 14 x 5.6
Machine scale 0.28
Glass wall at X ≈ 6
Rug and beanbags
Pinball
+Y
+X
Y = -7

Figure 15.1: Schematic plan of the gaming lounge and machine placement.

Table 15.1: Selected room-world object positions.

Object	Position or bounds
Floor / ceiling	Floor Z = 0; ceiling centered Z = 5.7
Television screen	(1.65,6.205,2.55)
Console tabletop	(1.65,5.4,0.96)
Rug	(1.25,1.3,0.018)
Beanbag centers	(0.10,0.75,0.43); (2.55,0.10,0.43)
Refrigerator body	(-4.66,6.1,1.5)
Fan center	(2.3,4.1,5.07)
Light / fan switch	(4.8,6.54,2.15); (5.35,6.54,2.15)

15.2 Ray Scene Snapshot and BVH

GameHub::rayScene creates a self-contained snapshot of room scenery, glass, switches, fan parts and the placed machine. Every triangle position and normal is transformed into room world space. TextureData is shared immutably; worker threads do not read changing GameObjects or OpenGL resources.

The verified initial scene export contains 27,870 nondegenerate triangles, 83,610 per-triangle corner records and 322 material entries. buildBVH creates 16,383 nodes for that snapshot. Each leaf holds at most four triangles.

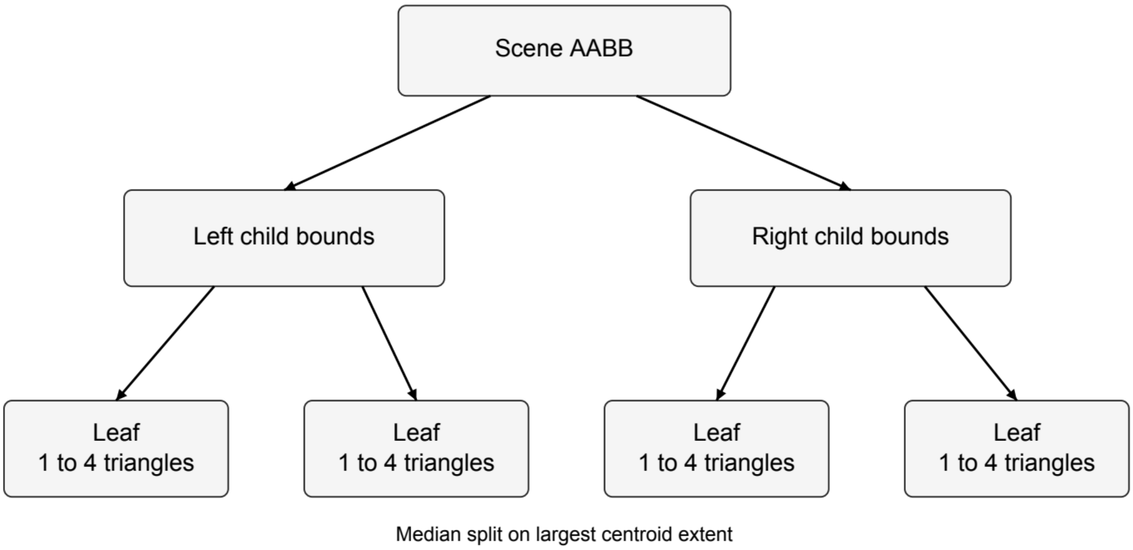

Scene AABB
Left child bounds
Right child bounds
Leaf
1 to 4 triangles
Leaf
1 to 4 triangles
Leaf
1 to 4 triangles
Leaf
1 to 4 triangles
Median split on largest centroid extent

Figure 15.2: Bounding volume hierarchy used to accelerate triangle intersection.

The builder computes triangle centroids, finds the axis with the largest centroid extent and partitions around the median using nth_element. It recursively constructs two child bounds. This is a balanced median-split BVH, not a surface-area-heuristic builder.

$$\text{centroid} = (v_0 + v_1 + v_2)/3 \tag{15.1}$$

Traversal first tests the root AABB. When both children intersect, the nearer child is processed first. A closer triangle hit shortens the current maximum distance, allowing farther queued bounds to be skipped. The stack contains node indices and their ray entry distances.

15.3 Ray Box and Triangle Intersections

A ray is r(t) = o + td with normalized direction d. AABB slab tests compute entry and exit distances on each axis and intersect the three intervals. Near-parallel directions are treated separately to avoid division by an almost-zero component.

$$t_{axis,0} = (min_{axis} - o_{axis})/d_{axis} \; ; \; t_{axis,1} = (max_{axis} - o_{axis})/d_{axis} \qquad (15.2)$$

RayTracer.cpp uses a two-sided Möller-Trumbore triangle test. For triangle vertices v_0,v_1,v_2, form edges $e_1 = v_1 - v_0$ and $e_2 = v_2 - v_0$. The determinant identifies parallel or degenerate configurations:

$$p = d \times e_2 \; ; \; D = e_1 \cdot p \; ; \; s = o - v_0 \qquad (15.3)$$

$$u = (s \cdot p)/D \; ; \; q = s \times e_1 \; ; \; v = (d \cdot q)/D \qquad (15.4)$$

$$t = (e_2 \cdot q)/D \; ; \; \text{barycentric weights} = (1-u-v, u, v) \qquad (15.5)$$

A hit requires $0 \le u \le 1$, $0 \le v$ and $u+v \le 1$. The ray distance must be above the minimum offset and below the current closest hit or the bounded shadow segment. The test rejects $|D| < 10^{-10}$ but accepts both determinant signs, so it is two-sided.

15.4 Surface Attributes at a Hit

$$\text{attribute} = (1-u-v)\text{attribute}_0 + u \; \text{attribute}_1 + v \; \text{attribute}_2 \qquad (15.6)$$

Normals, colors and UVs are barycentrically interpolated. The shading normal is normalized and oriented consistently with the geometric side being hit. Position is o + td. Texture sampling then supplies albedo for direct and recursive shading.

15.5 Shadows and Mirror Reflection

The ray tracer calculates direct Phong illumination on opaque surfaces. Each enabled point or spot light receives a shadow segment ending just before the light. An opaque intersection blocks direct diffuse/specular light. Transparent intersections reduce and tint visibility; geometry beyond the light cannot shadow that point.

$$\text{shadow segment: } \varepsilon \leq t < |\text{lightPosition-p}| - \varepsilon \tag{15.7}$$

Ambient remains an approximation rather than a visibility-integrated indirect-light result. For transparent shadow paths, the implementation walks up to 32 layers and multiplies tint, the square root of transmission and a Fresnel interface factor. This approximates straight-path light visibility through panes rather than tracing refracted shadow paths.

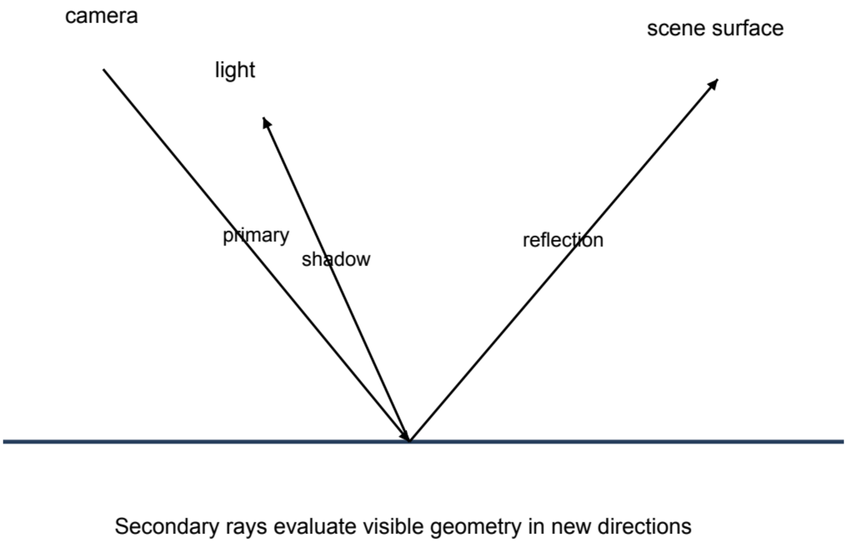


Figure 15.3: Primary, bounded shadow and recursive reflection rays.

$$d_{\text{reflection}} = d - 2(d \cdot N)N \tag{15.8}$$

Opaque material reflection mixes the local result with a recursive ray. The steel pinball uses reflection 0.72, the plunger 0.40, the tiled floor 0.18 and the kettle 0.50. These rays intersect actual geometry, including objects outside the primary camera view.

New ray origins move by $\varepsilon = 0.0001$ along the outward geometric normal, choosing the side using the outgoing direction. This reduces immediate self-intersection without changing the primary scene geometry.

15.6 Refraction Fresnel and Total Internal Reflection

Glass uses refractive index 1.5 by default. A geometric normal determines entry from air or exit back into air. The refracted ray follows Snell's law:

$$n_1 \sin \theta_1 = n_2 \sin \theta_2 \tag{15.9}$$

$$\eta = n_1/n_2 \; ; \; d_{transmitted} = \mathrm{refract}(d,N,\eta) \tag{15.10}$$

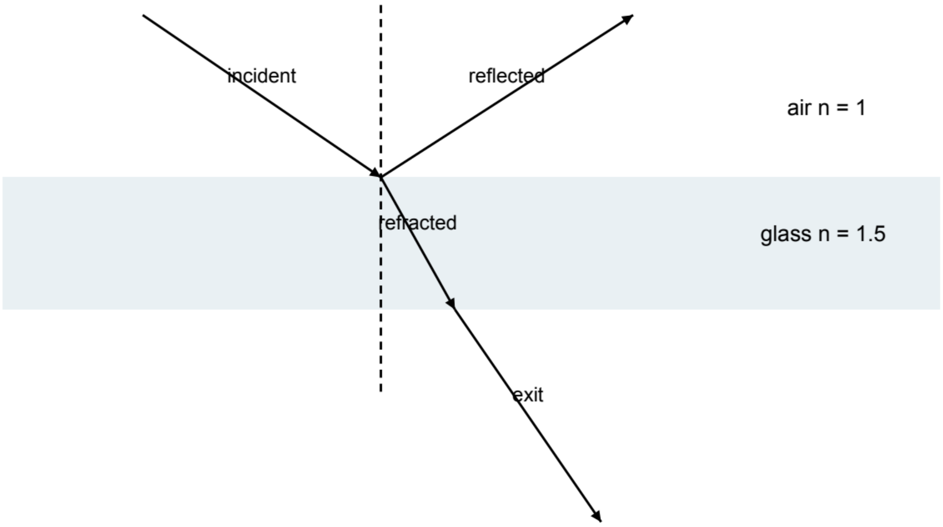


Figure 15.4: Reflection and refraction at the two interfaces of a glass pane.

The Fresnel function computes the average of squared parallel and perpendicular amplitude ratios, rather than the Schlick approximation. At normal incidence from air into index 1.5 glass, reflectance is $((1-1.5)/(1+1.5))^2 = 0.04$.

$$F = (r_{parallel}^2 + r_{perpendicular}^2)/2 \tag{15.11}$$

$$weight_{transmitted} = (1-F) \sqrt{}transmission \tag{15.12}$$

When $\eta \sin \theta_1 \geq 1$, total internal reflection gives F = 1 and no transmitted branch. For glass-to-air, the critical angle is asin(1/1.5) ≈ 41.81°. Ray recursion stops after six secondary levels or when the branch contribution is at most 0.005. Nested dielectric media and volumetric absorption are outside the current model.

15.7 Ray Traced Scene Results

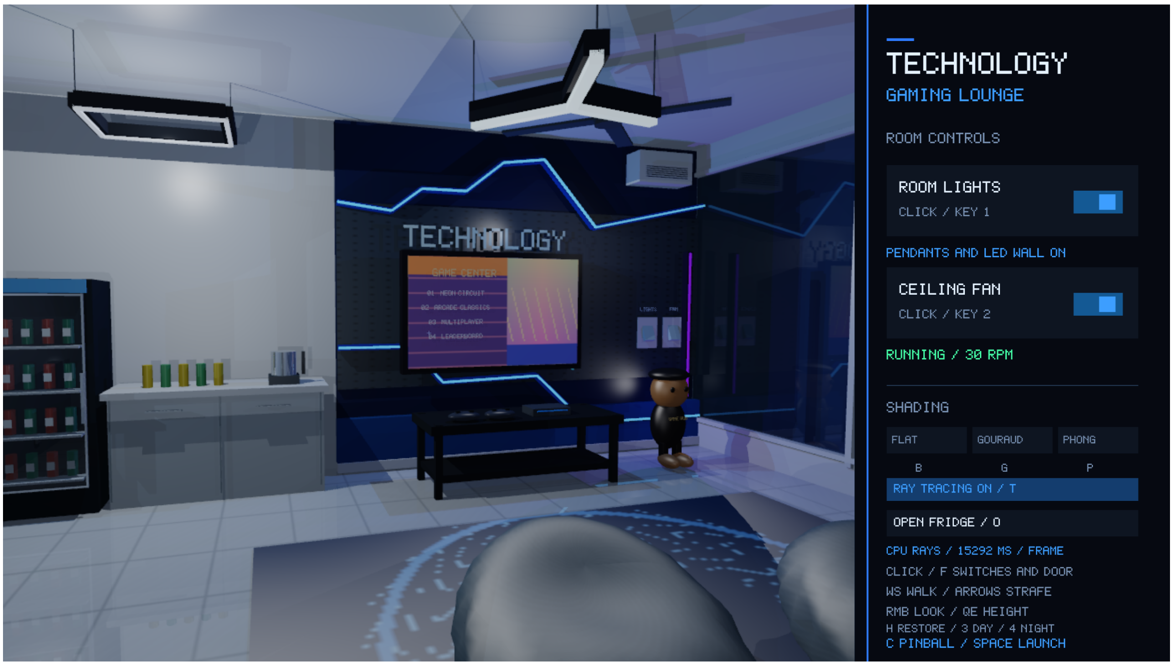

TECHNOLOGY
GAMING LOUNGE
ROOM CONTROLS
ROOM LIGHTS
CLICK / KEY 1
PENDANTS AND LED WALL ON
CEILING FAN
CLICK / KEY 2
RUNNING / 30 RPM
SHADING
FLAT
GOURAUD
PHONG
B
G
P
RAY TRACING ON / T
OPEN FRIDGE / O
CPU RAYS / 15292 MS / FRAME
CLICK / F SWITCHES AND DOOR
WS WALK / ARROWS STRAFE
RMB LOOK / QE HEIGHT
H RESTORE / 3 DAY / 4 NIGHT
C PINBALL / SPACE LAUNCH

Figure 15.5: Fresh room ray-tracing capture at a 960-pixel ray width and four samples per pixel.

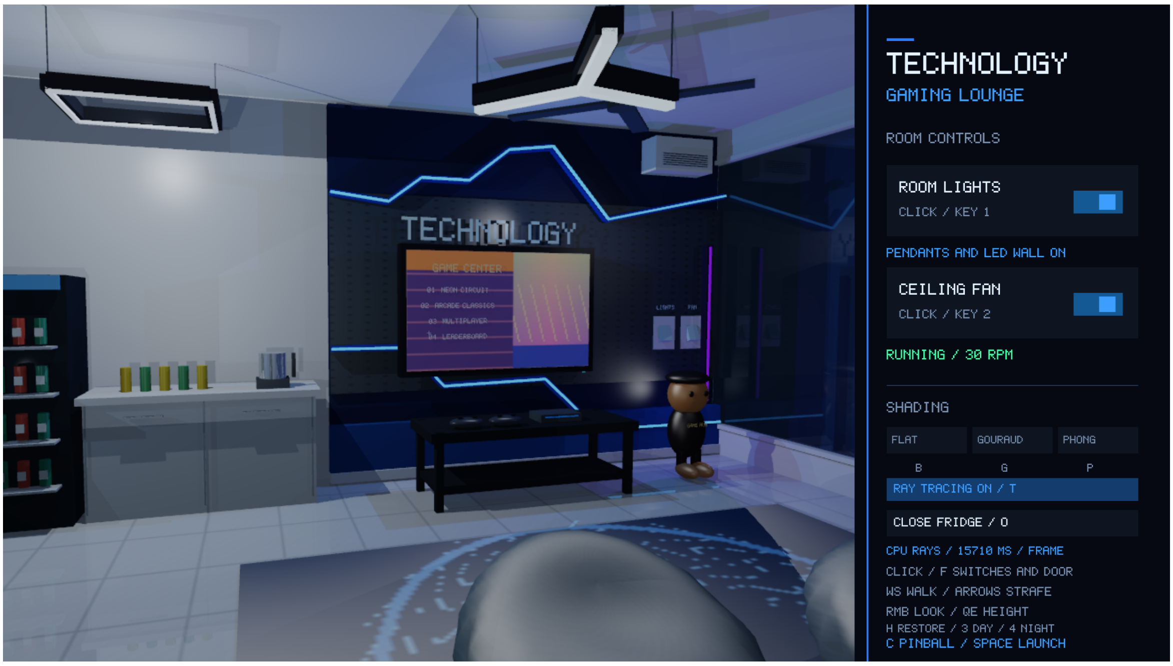

TECHNOLOGY
GAMING LOUNGE
ROOM CONTROLS
ROOM LIGHTS
CLICK / KEY 1
PENDANTS AND LED WALL ON
CEILING FAN
CLICK / KEY 2
RUNNING / 30 RPM
SHADING
FLAT
GOURAUD
PHONG
B
G
P
RAY TRACING ON / T
CLOSE FRIDGE / O
CPU RAYS / 15710 MS / FRAME
CLICK / F SWITCHES AND DOOR
WS WALK / ARROWS STRAFE
RMB LOOK / QE HEIGHT
H RESTORE / 3 DAY / 4 NIGHT
C PINBALL / SPACE LAUNCH

Figure 15.6: Actual ray-tracing capture with the refrigerator door open.

Both images were captured from the current executable during report preparation. The ray settings are width 960 and four samples per pixel. The glass, frame and handle share the hinge state exported for each capture.

15.8 Live Rendering and Quality Controls

RayTraceView launches a background scene job and uploads completed RGB pixels to an OpenGL texture. The OpenGL thread draws that texture and overlays the normal HUD. Live mode defaults to a 256-pixel-wide single-sample preview; captures default to width 480 and four samples. A larger width improves detail at greater CPU cost.

$$\text{rayImageHeight} = \text{round}(\text{rayWidth} \times \text{viewportHeight} / \text{viewportWidth}) \qquad (15.13)$$

Up to eight CPU workers take rows from an atomic counter. Each pixel uses deterministic sample offsets, so the same scene and settings produce identical pixels regardless of worker count. Samples are averaged before exposure and tone mapping.

Camera position, direction, FOV, viewport size, light/day mode, door target and camera choice invalidate outdated jobs. When no current completed image is ready, the application displays the raster scene. While ordinary animation advances, a completed ray snapshot may remain visible until the next background result arrives.

Table 15.2: Ray-rendering controls and implementation limits.

Item	Behavior
T or console ray button	Toggle ray display; B/G/P return to raster
--ray-width	Capture/live width clamped to 64-1920
--ray-samples	Capture samples clamped to 1-64
--capture-live	Exercise asynchronous rendering during capture
Illumination	Direct light plus mirror/dielectric recursion; ambient approximation
Transport exclusions	No diffuse global illumination, rough-reflection sampling or nested media

The renderer is a CPU Whitted-style implementation. It does not claim hardware-accelerated ray tracing or physically based path tracing. Its image time is displayed by the HUD; the report does not infer a general FPS benchmark from one capture.

Chapter 16

Interaction and Runtime Controls

Table 16.1: Runtime keyboard and mouse controls from main.cpp.

Input	**Action**
Space	Launch a READY ball at speed 18; switch to table view for a valid launch.
Enter / keypad Enter	Hold to charge plunger; release to launch a READY ball.
A / D	Power left / right flipper while held.
R	Reset score, lives, bonus, bank and ball; preserve session best.
C	Toggle room and table cameras.
W / S	Move active camera forward / backward.
Left / right arrows	Strafe active camera left / right.
Q / E	Move active camera upward / downward.
Right mouse drag	Look around with active camera.
Scroll	Change active camera field of view.
H	Restore default room composition and select room camera.
1 / 2	Toggle room lighting / fan power.
3 / 4	Select day / night mode.
B / G / P	Select Flat / Gouraud / Phong raster mode.
T	Toggle CPU ray tracing.
O	Toggle refrigerator-door target between closed and open.
F	Use the crosshair ray to activate a room switch or door.
Left click	Activate room console button or pick room switches/door.
Escape	Exit the application.

16.1 Console Buttons and Scene Picking

One-shot actions use an edge latch: a held key fires once until it is released. Flipper and plunger controls are continuous states. Room console button hits are checked before a scene picking ray is generated, so clicking a visible control chooses its direct action.

The HUD uses a 320 x 720 design space. Its scale is the smaller of consoleWidth/320 and framebufferHeight/720. It is centered inside the approximately 27%-wide console region. Mouse coordinates are converted from window units to framebuffer pixels and then into this design space.

$$s = \min(0.27W/320, H/720) \tag{16.1}$$

For wall switches, slab boxes are tested in room space. For the refrigerator door, inverse model transformation converts the world ray into the moving door frame. The origin uses $w = 1$ and direction $w = 0$. In table-camera mode, room scene picking is disabled.

Table 16.2: Room console action IDs.

ID	Action
1 / 2	Room light / fan toggle
3 / 4 / 5	Flat / Gouraud / Phong
6	Ray tracing toggle
7	Door open / close toggle

The pinball console reports session best, ball count, multiplier, pending bonus, recent event and target-bank state. The room console reports light state, fan RPM, day/night setting, shader choice, door angle and ray frame time. Both layouts are verified at multiple framebuffer sizes.

Chapter 17

Rendering Flow

The raster render loop resets depth/blend state, clears the framebuffer, selects the shader, sets the 3D viewport and uploads the active camera matrices. GameHub provides lights and scene ambient, draws room objects, applies the machine placement, draws the pinball machine and then renders transparent room glass.

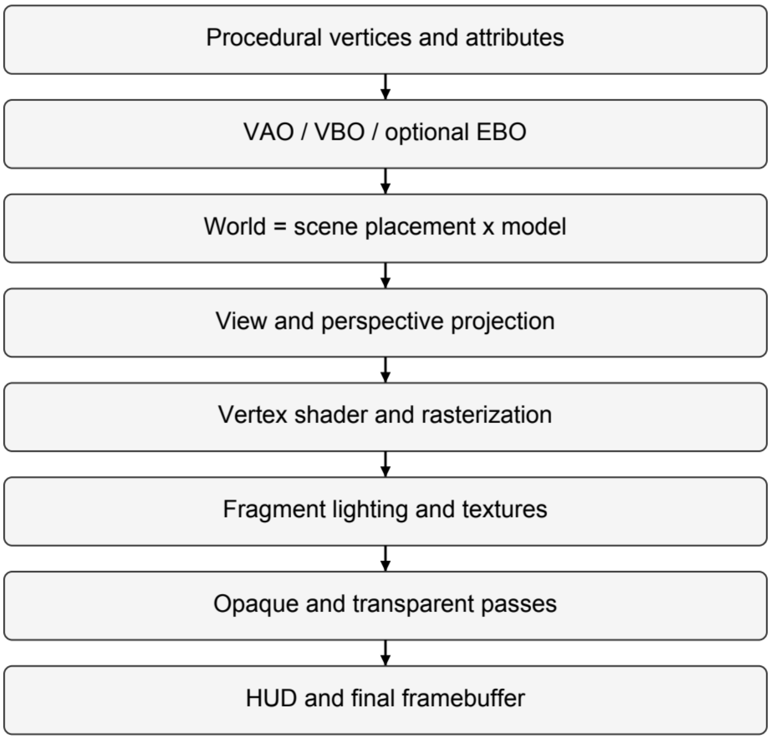

Procedural vertices and attributes
VAO / VBO / optional EBO
World = scene placement x model
View and perspective projection
Vertex shader and rasterization
Fragment lighting and textures
Opaque and transparent passes
HUD and final framebuffer

Figure 17.1: Raster pipeline from generated vertices to the final framebuffer.

$$p_{clip} = P\ V\ M_{scene}\ M_{object}\ p_{local} \tag{17.1}$$

The GPU clips primitives, rasterizes triangle coverage, interpolates attributes and runs the fragment shader. The viewport maps normalized device coordinates into framebuffer pixels. Finally the viewport expands to the complete window so the HUD can draw its console.

17.1 Opaque Transparent and HUD Passes

Opaque objects write depth normally. The machine cover enables blending with source-alpha and one-minus-source-alpha factors while disabling depth writes. GameHub draws room glass after opaque scenery in descending camera-distance order, also with depth writes disabled. These passes then restore the relevant state.

$$C_{\text{blend}} = \alpha C_{\text{source}} + (1-\alpha)C_{\text{destination}} \tag{17.2}$$

The transparent sorting uses object-center distance, not per-triangle order. It is sufficient for the selected room arrangement but is not a general order-independent-transparency algorithm. Ray mode resolves glass through actual ray intersections and refraction, so its result follows a different transport path.

17.2 Procedural Textures

Mesh::loadTexture generates RGB textures in memory. The board is 512 x 1024; other named textures are 128 x 128. The artwork includes navy circuits, purple lower-board rays and a cyan reactor. Room textures create white grout tiles, acoustic holes, woven upholstery, electric rug patterns and television color regions.

Raster texture wrap is repeat; minification is linear mipmap-linear and magnification is linear. The ray sampler retains the same RGB data and applies periodic bilinear filtering. It does not sample a mipmap pyramid, so ray and raster minification are not strictly identical.

17.3 Ray Presentation Pass

A completed CPU image uploads as GL_RGB8 and is shown through ray_present shaders with a generated full-screen triangle. The HUD remains an OpenGL overlay. A synchronous capture traces the final simulated frame, while --capture-live waits for a completed background frame.

The program locates shaders beside the executable and requests an OpenGL 3.3 core context with four-sample multisampling. These choices make launch behavior and screenshots reproducible from the project directory or the launcher.

Chapter 18

Implementation Summary and Validation

Table 18.1: Mathematics mapped to the principal implementation.

Concept	Implemented method	Source location
Cuboid	Eight corners expanded into 36 records; face normals.	Mesh.cpp - createCube
Plane	Four XY corners; six expanded records.	Mesh.cpp - createPlane
Cylinder	Circular rings, side strips and cap fans.	Mesh.cpp - createCylinder
Sphere	Latitude/longitude grid and two indexed triangles per cell.	Mesh.cpp - createSphere
Ring	Two angular parameters; indexed torus grid.	Mesh.cpp - createRing
Flipper mesh	Cuboid body plus sampled upper cap patches.	Mesh.cpp - createFlipper
Beanbag	Deformed sphere, seat depression and accumulated normals.	GameHub.cpp - beanbagMesh
Model transform	Translation x quaternion rotation x scale.	Transform.cpp
Machine placement	Room translation x uniform 0.28 scale.	GameHub.cpp - initialize
Board constraint	$z(y)=1.4+y\tan 8°$; vertical radius seating.	Playfield.cpp; PinballMachine.cpp
Flipper pivot	Pivot translate, Z rotate and half-length offset.	Flipper.cpp - getModelMatrix
Door hinge	Hinge plus rotated stored part offsets.	GameHub.cpp - updateDoor
Game dynamics	240 Hz accumulator and projected downhill acceleration.	PinballMachine.cpp - physicsStep

18.1 Implementation Map Continued

Table 18.2: Interaction rendering and ray-tracing implementation map.

Concept	Implemented method	Source location
Contact	Circle/rectangle and closest-point capsule tests.	Bumper.cpp; Target.cpp; PinballMachine.cpp
Scoring	Cooldown events, bank multiplier and drain bonus.	PinballMachine.cpp
Camera	Z-up yaw/pitch basis, LookAt and Perspective.	Camera.cpp
Picking	Inverse projection/view; ray slab tests and door local frame.	main.cpp; GameHub.cpp
Lighting	Phong reflection vectors, distance attenuation and spot cone.	phong.frag; gouraud.vert
Flat mode	Per-fragment ambient and diffuse without specular.	flat.vert; flat.frag
Normal matrix	Inverse transpose of combined transform.	Vertex shaders; RayTracer.cpp
Textures	Procedural RGB artwork; raster mipmaps and CPU bilinear sampling.	Mesh.cpp
Ray acceleration	Median-split triangle BVH; near-first traversal.	RayTracer.cpp - buildNode/intersect
Ray transport	Bounded shadows, mirror rays, Snell and exact dielectric Fresnel.	RayTracer.cpp - trace/fresnel
Async display	Immutable snapshot, background job, RGB texture presentation.	RayTraceView.cpp
HUD	Scalable bitmap text, score console and buttons.	HUD.cpp
Startup and capture	GLFW context/input, shader loading and PPM framebuffer capture.	main.cpp

18.2 Build and Regression Results

The current project was rebuilt and tested during report preparation on 8 October 2026. CMake completed the core library, executable and test targets. CTest reported three passing suites and zero failures.

Table 18.3: Observed regression-test run.

CTest suite	Result	Observed time	Main coverage
pinball_physics	PASS	1.70 s	12 launch combinations, charging, scoring, save, lives, flippers, frame-rate consistency
arena_controls	PASS	0.51 s	Switch picking, independent controls, fan coast, door reversal, bounds, HUD scaling
raytracing	PASS	1.03 s	BVH reference agreement, normals, shadows, reflection, refraction, Fresnel/TIR, deterministic workers

The twelve launch combinations cover render intervals for 30, 60, 144 and 10 FPS with launch speeds 14, 18 and 20. The scoring scenario verifies 1400 after the first bank and 1800 after its multiplied drain bonus. Ray tests compare closest hits to an independent exhaustive reference and compare single-worker and four-worker RGB results.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j4
ctest --test-dir build --output-on-failure
```

18.3 Rendered Capture Verification

A fresh table capture completed with score 450 and state IN_PLAY. A fresh room capture completed with score 0 and state READY. Shader programs loaded successfully in both runs. A high-detail room ray capture was also generated with ray width 960 and four samples per pixel.

That ray capture traced a 960 x 740 image over 27,870 triangles and 16,383 BVH nodes, reporting 15,292.2 ms for the CPU ray frame. This is one observed local capture measurement at those settings.

The test times describe one local validation run, not performance targets. Screenshots establish representative output and shader execution; the headless suites establish the tested logic properties. No unsupported hardware-wide performance claim is made.

Chapter 19

Conclusion

Interactive 3d Pinball Game demonstrates a complete graphics application around a playable machine and an interactive room. Its visible objects originate in procedural mesh equations, then pass through object and scene transforms, camera matrices, lighting, material properties and rasterization. The alternate CPU renderer uses the same scene state to trace shadows, reflections and glass transport.

The strongest connection between mathematics and behavior is the shared geometry/state path. The board-height function seats the ball and components on the slope; rail endpoints define both contact and visible guides; flipper pivots and angles drive both rendering and moving contact response; hinge offsets preserve the refrigerator assembly; and exported world-space triangles reproduce the scene for ray workers.

Fixed 240 Hz physics separates gameplay contact resolution from the display frame rate. The game-state rules make launches, scoring, ball saves, bonus collection and three-ball accounting explicit. Independent room controls and movable cameras provide direct demonstrations of animation, picking, attenuation and shading.

19.1 Limitations and Future Work

The physics is a constrained XY model with vertical surface seating. It could be extended with rolling angular velocity, free-flight motion and continuous swept collision tests. The Flat program could be revised to use true per-face constant normals and illumination. Shared GPU mesh instances could reduce repeated primitive storage.

The ray tracer could improve with a surface-area-based BVH, progressive accumulation, rough reflections, diffuse indirect transport and more complete dielectric-medium tracking. Transparency sorting and furniture-aware camera collision could also be generalized. These are future extensions; they are not presented as features of the submitted implementation.

The completed build, three regression suites, actual render captures and full geometry archive provide concrete evidence for the behavior documented in this report.

References

[1] Project source. Current C++ headers and implementation files in include/ and src/, GLSL files in shaders/, tests/ and CMakeLists.txt. Primary evidence for all implementation-specific coordinates, equations, parameters and behavior in this report.

[2] The Khronos Group. *The OpenGL Graphics System: A Specification, Version 3.3 Core Profile.* Reference for vertex specification, buffers, shader execution, rasterization and blending.
registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf

[3] GLFW. *GLFW 3.3 Documentation.* Reference for window/context creation and keyboard, mouse and framebuffer handling.
www.glfw.org/docs/3.3/

[4] Pharr, Jakob and Humphreys. *Physically Based Rendering, fourth edition, Bounding Volume Hierarchies.* Background on spatial acceleration; the project's actual builder uses median splitting.
pbr-book.org - Bounding Volume Hierarchies

[5] Pharr, Jakob and Humphreys. *Physically Based Rendering, fourth edition, Specular Reflection and Transmission.* Background for ideal specular and dielectric transport.
pbr-book.org - Specular Reflection and Transmission

[6] GLM. *OpenGL Mathematics manual.* Background for vector, matrix, quaternion, LookAt and perspective utilities.
github.com/g-truc/glm/blob/master/manual.md

Report Preparation

The supplied robotic-surgery report was used as a structural and visual reference. Project-specific prose, tables and illustrations were rewritten for this pinball implementation. OpenAI Codex assisted source inspection, report organization, geometry export and layout verification. Actual project captures supply the application images; technical diagrams are explanatory drawings.

Appendix A Coordinate Registry

A.1 Bumpers and Targets

The table gives component center heights after the update seats the bodies on the board. Main and side bumper bodies both have height 0.45, so their centers use z(y)+0.225. Targets use z(y)+0.22.

Table A.1: Exact object placement registry rounded to six decimals.

Object	**X**	**Y**	**Center Z**	**Radius / points**
Bumper 0	-1.150000	3.150000	2.067704	0.42 / 100
Bumper 1	0.150000	4.150000	2.208244	0.42 / 150
Bumper 2	1.350000	3.050000	2.053650	0.42 / 100
Bumper 3	-2.450000	0.350000	1.674189	0.28 / 50
Bumper 4	-2.550000	-0.750000	1.519595	0.28 / 50
Bumper 5	-2.250000	-1.750000	1.379054	0.28 / 50
Target 0	-1.650000	4.900000	2.308650	100 points
Target 1	1.950000	4.550000	2.259461	100 points
Target 2	0.950000	1.850000	1.880001	100 points
Target 3	-0.150000	2.050000	1.908109	100 points

A.2 Ready Ball and Flipper Tips

At Y = -4.6, surface Z is 0.753512. The ready-ball center is (2.95,-4.6,1.033512). Each flipper pivot Z is 0.873512. With length 1.48, the left rest tip is approximately (-0.443238,-5.294818,0.873512), and the right rest tip is (0.443238,-5.294818,0.873512).

At the active stroke, the left tip is approximately (-0.359255,-4.093810,0.873512), and the right tip is (0.359255,-4.093810,0.873512). These segment tips are contact geometry; the rendered cap patches extend beyond or above portions of the body.

A.3 Complete Rail Endpoint Registry

Table A.2: All eleven physical and visible rail segments.

ID	A X,Y	B X,Y	Radius	Bounce	Type
0	(2.42,-6.00)	(2.42,4.25)	0.08	0.45	Guide
1	(3.42,4.65)	(2.35,5.78)	0.08	0.85	Guide
2	(2.35,5.78)	(0.20,5.78)	0.08	0.80	Guide
3	(-3.30,3.80)	(-2.90,5.35)	0.09	0.65	Guide
4	(-2.90,5.35)	(-1.30,5.78)	0.09	0.65	Guide
5	(-3.30,-2.40)	(-2.60,-3.90)	0.09	0.55	Guide
6	(-2.60,-3.90)	(-1.75,-4.60)	0.09	0.55	Guide
7	(2.30,-2.40)	(2.15,-3.90)	0.09	0.55	Guide
8	(2.15,-3.90)	(1.75,-4.60)	0.09	0.55	Guide
9	(-2.50,-2.45)	(-1.05,-3.95)	0.10	0.85	Sling
10	(1.65,-2.45)	(0.95,-3.95)	0.10	0.85	Sling

To generate every rail corner, promote each endpoint to 3D with Z = 1.4 + Y tan8° + 0.18. Compute the side vector from Equation 5.23. The lower four corners are A-side-Zoffset, A+side-Zoffset, B+side-Zoffset and B-side-Zoffset. The upper four add Zoffset, where Zoffset = (0,0,0.18).

A.4 Cube Corner Rule for All Box Objects

Every createCube(w,h,d) follows Table 5.1 and Table 5.2. A full world-space record is obtained by applying the object model and, for machine objects, the placement matrix. The companion scene CSV includes all visible nondegenerate triangles from boxes, custom meshes and curved primitives. It is therefore the exhaustive numerical supplement to the concise coordinate tables.

Values in the prose and tables are rounded mathematical evaluations. The CSV stores nine significant digits from the actual single-precision C++ mesh and transform computations. Small discrepancies in the final displayed digit can arise from float trigonometry.

Appendix B Complete Vertex Archive

The companion archive, **Interactive_3d_Pinball_Geometry.zip**, contains the exact exported records. CSV files can be opened in a text editor, spreadsheet application or analysis tool. Row IDs are zero-based. The report provides their definitions so each number has an interpretable coordinate frame and attribute meaning.

Table B.1: Geometry archive inventory.

File group	Records	Coordinate frame / purpose
cube_unit_vertices / triangles	36 vertices; 12 triangles	Local unit cuboid with exact record ordering
plane_unit_vertices / triangles	6 vertices; 2 triangles	Local unit plane; repeat 1
main_bumper_cylinder	336 vertices; 112 triangles	Local radius 0.42, height 0.45, 28 segments
ball_sphere	551 vertices; 1008 triangles	Local radius 0.28; includes submitted pole triangles
flipper	1764 vertices; 588 triangles	Local body and both cap patches
reactor_ring	369 vertices; 640 triangles	Local torus with R 0.68 and r 0.04
screen	4 vertices; 2 triangles	Local 2.98 x 1.77 television quad
complete_scene_vertices.csv	83,610 corner records	Room-world startup scene; 27,870 nondegenerate triangles
scene_materials.csv	322 material entries	Material properties indexed by material_id
summary.json; README.txt	Metadata and schema	Counts, snapshot conditions and attribute definitions

B.1 Export Schema and Snapshot Conditions

Primitive vertex columns are vertex_id, x,y,z, nx,ny,nz, r,g,b, u,v. Triangle files list triangle_id and the three vertex IDs. The complete scene instead lists triangle_id, corner 0-2, material_id and the same attributes. Repeated positions are retained intentionally because normals and UVs can differ at a seam or hard edge.

The export constructs GameHub without an OpenGL window, switches fan power off before update(0), leaves the door closed, keeps the ball READY and uses night mode with lights on. Scene topology and materials are exported from the same CPU pathway used by the ray tracer. It is one precise startup snapshot, not all possible animated states.